



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Tecniche di IA explainability per la predizione del diabete

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Vincenzo Danese

Matricola: 0512107959

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

«La vittoria più importante è quella che devo ancora conseguire» (Enzo Ferrari)
*«I want to dedicate this to the losers... out of losing i always learned more... so i got stronger
in losing» (Niki Lauda, 2016)*
Alla mia famiglia, alla mia ragazza, a Viktor e Victoria.

Abstract

L'uso di applicativi di intelligenza artificiale in contesti reali porta con sé varie problematiche; in questo studio verranno analizzate come esse impattino in ambito medico. Nello specifico, si focalizza sui modelli predittivi per il diabete mellito e su come affrontare le criticità di interpretabilità e trasparenza tipiche di molti modelli di AI, con particolare attenzione alla responsabilità. Come si può affidare la vita delle persone a modelli matematici che effettuano scelte non trasparenti ed interpretabili? Per risolvere questo problema, sono state adottate alcune tecniche all'avanguardia che aiutano a comprendere il processo decisionale dei vari modelli e ad aumentare la nostra fiducia in essi, mediante una piattaforma software che integra diversi modelli di AI per predire il diabete, permettendo di confrontare e analizzare le scelte di ogni modello per ottenere prospettive diverse e un quadro più chiaro. Ottenendo così una piattaforma efficace nella predizione del diabete sia dal punto di vista delle performance, sia dal punto di vista della trasparenza.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	v
1 Introduzione	1
2 Obiettivi e metodologie usate	4
2.1 Obiettivi della ricerca	4
2.2 Dataset utilizzato	5
2.2.1 Data Understanding & Data Preparation	6
2.3 Model Evaluation	11
2.3.1 Metriche di performance usate	11
2.3.2 Strategie di validazione	13
2.4 Machine learner adottati	13
2.4.1 Random forest	14
2.4.2 XGboost	15
2.4.3 MLP	16
2.4.4 k-nearest neighbors	17
2.4.5 AdaBoostClassifier	18
2.4.6 Logistic regression	20
2.5 Tecniche di explainability	21

2.5.1	LIME	21
2.5.2	SHAP	22
3	Risultati	23
3.1	Random Forest	23
3.2	XGBoost	28
3.3	MLP	33
3.4	k-nearest neighbors	38
3.5	AdaBoostClassifier	42
3.6	Log Regression	48
3.7	Riepilogo risultati	52
4	Conclusioni	55
	Bibliografia	57

Elenco delle figure

2.1	Distribuzione delle feature numeriche.	6
2.2	Distribuzione delle feature numeriche mediante un pair plot.	8
2.3	Correlazione dei valori di glucosio e HbA1c per i casi di diabete . . .	9
2.4	Grafico che mostra come i casi di diabete si distribuiscono in relazione al sesso e allo stato di fumatore	10
2.5	Distribuzione dei casi di diabete per fasce di età e distinte per sesso .	11
3.1	Confronto delle prestazioni del modello con parametri di default sulla destra e usando max-depth 10 sulla sinistra usando la K-cross-validation	25
3.2	Grafico lime che mostra una predizione etichettata come non diabetico	26
3.3	Grafico waterfall mostrante un falso negativo di random forest	26
3.4	Grafico waterfall mostrante un vero positivo di random forest	27
3.5	Heatmap shap rappresentante il comportamento del modello random forest	27
3.6	Grafico che mostra come cambia il valore auc-roc rappresentando valori più alti in gialli e valori inferiori tendenti al viola	28
3.7	Distribuzione delle metriche di xgboost usando k-cross-validation . .	29
3.8	grafico Lime per l'interpretazione delle scelte che hanno portato a classificare l'istanza corrente come positiva	30
3.9	Grafico waterfall mostrante un falso negativo di Xgboost	31

3.10	Grafico waterfall mostrante un vero positivo di Xgboost	31
3.11	Heatmap shap rappresentante il comportamento del modello Xgboost	32
3.12	Confronto delle prestazioni del modello con parametri empirici(destra) e parametri tramite random search(sinistra) usando la K-cross-validation	35
3.13	Grafico lime MLP	36
3.14	Grafico waterfall mostrante un falso negativo di MLP	36
3.15	Grafico waterfall mostrante un vero positivo di MLP	37
3.16	Heatmap shap rappresentante il comportamento del modello mlp . .	37
3.17	Analisi della distribuzione delle metriche usando cross validation. . .	38
3.18	knn lime	39
3.19	Grafico waterfall mostrante un falso negativo di KNN	40
3.20	Grafico waterfall mostrante un vero positivo di KNN	41
3.21	Heatmap shap rappresentante il comportamento del modello KNN .	41
3.22	Grafici 3d che rappresentano l'andamento delle metriche f1-score, recall, precision e accuracy, al variare della learning rate e n-estimators	43
3.23	Confronto delle prestazioni del modello con parametri di default sulla destra e usando max-depth 10 sulla sinistra usando la K-cross-validation	44
3.24	Distribuzione delle metriche usate su 10 iterazioni della k-cross-validation	44
3.25	Grafico mostrante le scelte di una predizione fatta con ADA tramite lime	45
3.26	Grafico waterfall di un falso negativo di Adaboost	46
3.27	Grafico waterfall di un vero positivo di Adaboost	47
3.28	Hetmap shap mostrante il comportamento globale di adaboost . . .	47
3.29	Visualizzazione della media delle metriche del modello usando k cross validation con k=10	48
3.30	Grafico mostrante le scelte di una predizione di log-regression tramite lime	49
3.31	Grafico waterfall mostrante un falso negativo di log-regression . . .	50
3.32	Grafico waterfall mostrante un vero positivo di log-regression	50
3.33	Heatmap shap mostrante le scelte generali del modello log-regression	51

Elenco delle tabelle

3.1	Prestazioni di random forest al variare del numero di alberi	24
3.2	Come si comporta random forest aumentando la lunghezza massima che più raggiungere	24
3.3	Tabella che riporta le performance del modello al variare del numero di nodi interni della rete neurale	33
3.4	Prestazioni di KNN al variare del numero dei vicini	38

CAPITOLO 1

Introduzione

Il diabete mellito è una delle patologie croniche più diffuse a livello globale [1]. La malattia è associata ad una serie di condizioni metaboliche legate a livelli alti di glicemia, causati da insufficienza parziale o totale dell'ormone peptidico prodotto al livello del pancreas, che prende il nome di insulina. Esso si può catalogare in quattro classi [1];

- **Diabete tipo 1:** una condizione autoimmune in cui, per fattori genetici o innescata da infezioni virali, il sistema immunitario attacca e distrugge le cellule β pancreatiche, determinando un deficit insulinico.
- **Diabete tipo 2:** la forma più comune, caratterizzata dalla resistenza all'insulina e da una progressiva disfunzione delle cellule β , che porta a una gestione inefficace del glucosio.
- **diabete gestazionale:** questa è una forma di diabete che si manifesta durante la gravidanza e, in genere, scompare dopo il parto. Tuttavia, se non ben gestito, può comportare rischi sia per la madre che per il feto.
- **Casi meno comuni:** sono associati a mutazioni genetiche o casi di tipo 2 meno frequenti, come il diabete ereditario.

Secondo l'OMS(Organizzazione Mondiale della Sanità), il numero di persone affette dal diabete è in costante crescita, con un aumento dal 7% nel 1990 al 14% negli adulti con età maggiore ai 18 anni, con tassi di mortalità anch'essi in aumento dal 2000 ad oggi.

Nel 2021 il diabete è stato la causa di morte diretta per 1.6 milioni di persone, con quasi il 50% di essi deceduti prima dei 70 anni di età. Con un altro mezzo milione di persone morte per problemi renali direttamente causati dal diabete [2]. Come osservato, il diabete è una patologia dal grosso impatto sulla salute umana, specialmente il diabete di tipo 2 che riguarda il 95% di tutti i casi di diabete presenti; vista la sua pericolosità, la prevenzione è fondamentale poiché se non curato in tempo può portare alla morte o alla perdita di qualche arto.

Come riportato dall' OMS nel suo report [2], il rilevamento del diabete tipo 2 può risultare ostico, poiché spesso i sintomi sono simili a quelli che si manifestano nel caso del tipo 1, ma sono leggermente marcati. Questa condizione porta spesso a un tardo rilevamento della malattia con diversi anni di ritardo, quando le complicazioni sono già in atto. Il diabete viene diagnosticato principalmente tramite analisi del sangue, da cui si possono analizzare i livelli di glucosio ed emoglobina glicosilata, anche abbreviata in HbA1c, e stabilire dato un range di valori accettabili se un paziente è o meno diabetico; la rivelazione può essere difficile nel caso di valori ai limiti di questi range. Si parla di persone sane con valori di glucosio a digiuno <6.1 mmol/l, a due ore dall'assunzione orale <7.8 mmol/l e valori di HbA1c di <39 mmol/l (5.7%). Le persone con diabete invece hanno valori di HbA1c superiori a $\geq 6.5\%$, glucosio a digiuno $\geq 7.0\%$ mmol/l e a 2h dal test orale di $\geq 11.1\%$ mmol/l. Tutta la zona intermedia è considerata zona prediabetica, come riportato nell'articolo [1].

Nella condizione di prediabetica i valori si avvicinano alle soglie limite; tuttavia, tali valori di confine possono altresì essere influenzati da altre patologie o condizioni cliniche, rendendo quindi la diagnosi di prediabetica particolarmente complessa.

Il presente lavoro si propone di sviluppare una piattaforma software atta a supportare il processo diagnostico del diabete mediante l'impiego di algoritmi di intelligenza artificiale, in particolare tecniche di machine learning (ML). Attraverso l'analisi di un dataset contenente variabili cliniche, quali i livelli di glucosio e l' HbA1c, si intende individuare in maniera tempestiva i casi di diabete.

Questa ricerca affronta la sfida rappresentata dall'utilizzo di modelli di intelligenza artificiale in un ambito critico come quello della diagnosi del diabete. In particolare, affidarsi a modelli "scatola nera" potrebbe portare a decisioni opache e potenzialmente rischiose, con conseguenze gravi per la salute dei pazienti. Per questo motivo, si integra nel sistema un approccio di explainability, che permette di:

- **Rendere trasparente il processo decisionale:** Le tecniche di explainability evidenziano le variabili e i criteri utilizzati dai modelli per giungere a una diagnosi, facilitando così l'interpretazione dei risultati.
- **Validare le scelte diagnostiche:** Attraverso l'analisi dei modelli, è possibile comprendere come essi reagiscono a variazioni degli input, anche quando alcune feature sono considerate meno rilevanti dal punto di vista clinico.
- **Confrontare modelli differenti:** L'implementazione di più modelli di machine learning consente di effettuare un confronto incrociato, garantendo una valutazione più robusta e affidabile delle decisioni prese.

Questo approccio multidimensionale non solo contribuisce a migliorare l'affidabilità del sistema, ma offre anche un importante supporto al personale medico, fornendo una visione chiara e giustificata dei processi diagnostici.

I risultati di questo studio hanno mostrato come tutti i modelli ML adottati valgiano le loro scelte dando maggior peso alle variabili chiave, quali glucosio ed emoglobina glicosilata, con ottime prestazioni e margini di miglioramento futuro su quasi tutti i modelli. Mostrando come l'approccio scelto possa aiutare a mitigare alcune criticità dei modelli AI.

Nei capitoli successivi saranno spiegate in dettaglio le scelte fatte, il dataset utilizzato, i modelli di IA utilizzati e come ognuno di essi affronta le proprie scelte mediante tecniche di explainability.

Obiettivi e metodologie usate

2.1 Obiettivi della ricerca

La presente ricerca ha come obiettivo la realizzazione di una piattaforma software finalizzata alla predizione del diabete, utilizzando modelli di intelligenza artificiale (IA) che siano sia performanti che trasparenti.

A tale scopo, lo studio si articola in tre sezioni principali:

- **Explainabilty.**

Per superare il problema della trasparenza decisionale tipico dei modelli IA definiti a “scatola nera”, verranno adottate metodologie interpretative avanzate quali **SHAP** e **LIME**. Queste tecniche permetteranno di esplicitare il ragionamento alla base delle predizioni, fornendo un supporto chiaro e giustificato alle decisioni del modello.

- **Confronto incrociato.**

Implementazione di molteplici modelli IA ci permettere di avere un responso diverso e quindi di avere più prospettive con cui confrontarsi e capirne il ragionamento fatto, così da poter rafforzarne la validazione.

- **Feature selection.**

L'impiego di tecniche di selezione delle caratteristiche si rivela fondamentale per identificare ed eliminare le variabili meno significative nella rilevazione del diabete. Tale operazione non solo migliora l'efficienza e la robustezza del modello, ma consente anche di analizzare in che modo le variazioni delle feature influenzino le performance predittive.

Un elemento essenziale per la realizzazione di modelli robusti e versatili è l'utilizzo di un dataset accuratamente ingegnerizzato. La tesi proseguirà con un'analisi dettagliata del dataset impiegato, evidenziandone le caratteristiche e il contributo al miglioramento complessivo delle previsioni.

2.2 Dataset utilizzato

Il dataset usato per questa ricerca è il risultato di un lavoro di tesi sulla predizione del diabete di uno studente dell'università degli studi di Salerno [3] .

I dati presenti in questo dataset sono i seguenti:

- **HbA1c:** valori di emoglobina glicosilata in percentuale % presenti nel sangue.
- **Glucosio:** valori di glucosio presenti nel sangue del paziente.
- **BMI:** l'indice della massa corporea di una persona, che ci fornisce informazione sul corpo della persona.
- **Ipertensione:** ci informa se una persona è affetta da ipertensione o meno.
- **Sesso:** indica il sesso della persona sottoposta a diagnosi.
- **Età**
- **Fumatore:** è un informazione riguardante lo stato di fumatore del paziente, indica se il paziente fuma, ha fumato in passato oppure non abbia mai fumato.
- **Diabete:** indica con valore 1 i pazienti diabetici e con 0 i non diabetici.

Il dataset in questione presenta feature molto rilevanti per la predizione del diabete, come riportato dallo studio [1].

I casi di diabete presentano una forte correlazione con i valori di glucosio, HbA1c; un legame con il peso, malattie cardiovascolari, avanzamento di età e un aumento dei rischi per le persone che fumano di divenire affette dal diabete.

2.2.1 Data Understanding & Data Preparation

In questa fase si procede con l'esame dettagliato del dataset selezionato, volto a comprendere le caratteristiche intrinseche delle variabili, le relazioni esistenti tra di esse e l'eventuale presenza di valori anomali. Il dataset in analisi è composto da 53.780 record, caratterizzati da un marcato sbilanciamento: 47.642 osservazioni appartengono alla classe dei non diabetici, mentre 6.138 alla classe dei diabetici. Un aspetto rilevante è che, essendo il dataset già sottoposto a un processo di ingegnerizzazione, non sono presenti dati mancanti né elementi di disturbo, garantendo così una buona qualità delle informazioni disponibili.

L'analisi della 2.1 permette di ricavare alcune considerazioni rilevanti sulle feature

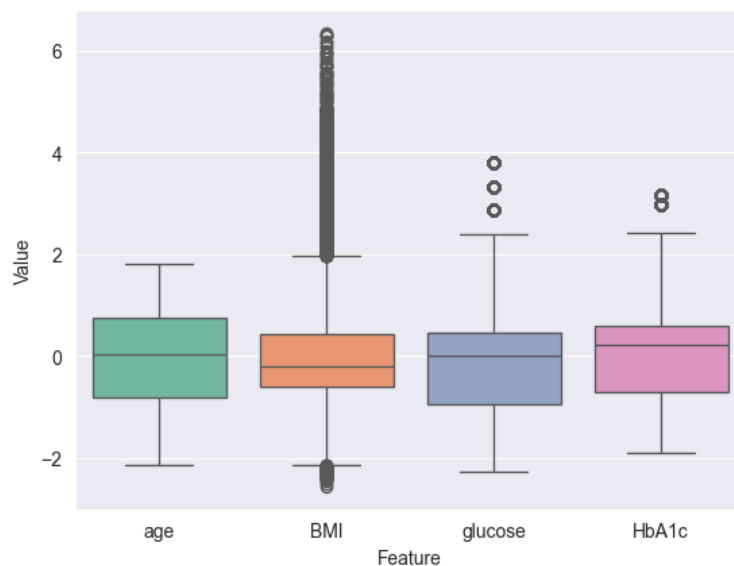


Figura 2.1: Distribuzione delle feature numeriche.

numeriche. In particolare, le variabili Age, Glucose e HbA1c presentano una distribuzione relativamente regolare, con il maggior numero di dati concentrato attorno ai

valori mediani e solo pochi rilevanti outlier, visibili in misura limitata soprattutto in **Glucose** e **HbA1c**.

In netto contrasto, la variabile BMI si caratterizza per una presenza significativa di valori estremi, che si discostano sensibilmente dalla media. Questa caratteristica della distribuzione del BMI potrebbe avere ripercussioni sulla capacità predittiva di modelli che sono particolarmente sensibili agli outlier. Tuttavia, il mantenimento di tali valori porta con sé un valore rappresentativo, in quanto riflette una porzione della popolazione reale che si colloca agli estremi del range di peso. L'approccio strategico prescelto prevede l'adozione di tecniche di scaling per normalizzare gli effetti degli outlier, nonché la possibilità di escludere temporaneamente la feature BMI durante la fase di predizione, qualora si riscontri un impatto negativo significativo. In definitiva, si è scelto di mantenere l'intera gamma dei dati, compresi quelli estremi, è motivata dalla volontà di preservare la reale variabilità della popolazione di riferimento.

Riassumiamo di seguito i motivi della nostra scelta:

- Dati di una parte della popolazione reale.
- Implementazione di modelli misti che siano più o meno robusti ai valori outlier.
- Feature scaling per i modelli sensibili ai valori outlier.
- La piattaforma implementa la possibilità di escludere dalla predizione, parte o tutte le features **non chiave**.

I modelli adottati, insieme alle tecniche pre-processuali messe in atto, hanno consentito di raggiungere performance ottimali, anche per quei modelli sensibili agli outlier, senza necessità di escludere i valori estremi presenti nella variabile **BMI**.

I modelli adottati e le relative performance saranno discussi nel corso di questo capitolo nelle sezioni seguenti.

Visualizing statistical relationships

Per favorire un'interpretazione intuitiva delle relazioni intercorrelate tra le variabili, si è impiegato il framework Seaborn, che offre strumenti grafici avanzati. L'analisi è stata integrata con la realizzazione di un pairplot, che ha permesso di

evidenziare le relazioni tra le variabili indipendenti, differenziando i dati in base alla variabile target (presenza o assenza di diabete).

In particolare, dalla Figura 2.2 emerge come il subplot dedicato alla variabile **Glucose**, posto lungo la diagonale, evidenzia un netto superamento di una determinata soglia nei casi positivi per il diabete. Analoga osservazione è valida per i valori di **HbA1c**. Un dato di particolare interesse è la tendenza dei soggetti positivi al diabete a distribuirsi in fasce di età sempre più elevate. La dettagliata analisi del subplot relativo alla correlazione tra **HbA1c** e **Glucose** ha, inoltre, contribuito a evidenziare come tali variabili possano rappresentare dei buoni indicatori nella distinzione dei casi diabetici da quelli non diabetici.



Figura 2.2: Distribuzione delle feature numeriche mediante un pair plot.

Andando nel dettaglio del grafico in **Figura 2.3**, che mette in correlazione il **Glucosio** e **HbA1c** per casi di diabete, si osserva, come, al crescere delle variabili

di **Glucosio** e **HbA1c** vi sia un aumento dei casi di diabete, evidenziando una correlazione tra le due variabili. Un aspetto di particolare rilievo è l'assenza di valori di **HbA1c** inferiori a 5,8 quando il **Glucosio** supera la soglia di 200; in modo analogo, per valori di **HbA1c** superiori a 6,8 non si riscontrano livelli di **Glucosio** al di sotto di 130. Se si suddivide il grafico in quattro regioni, simile a un piano cartesiano, le aree qui descritte ricadono nel secondo e nel quarto quadrante. Tutti questi aspetti ci portano a dire che le feature **Glucose** e **HbA1c** rappresentano le feature che chiameremo **chiave** nel corso di questa tesi, poiché sono le più discriminatorie per poter predire correttamente i casi di diabete.

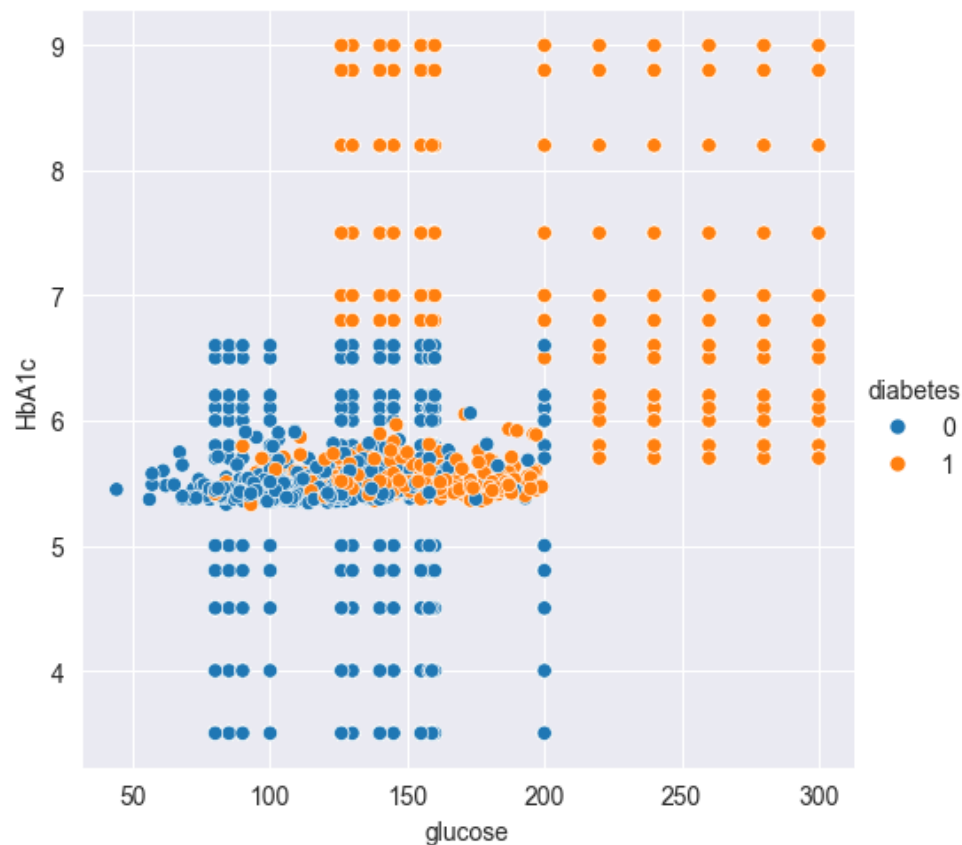


Figura 2.3: Correlazione dei valori di glucosio e HbA1c per i casi di diabete

Abbiamo esaminato i grafici che fanno riferimento alle variabili non numeriche, selezionando alcune tra le relazioni più interessanti.

In **Figura 2.4** viene mostrata la dipendenza del diabete dal **Gender** e **Smoking status**, si nota come il genere femminile tenda ad essere incline al diabete per i non fumatori, ma dal grafico si evince una forte resistenza al fumo.

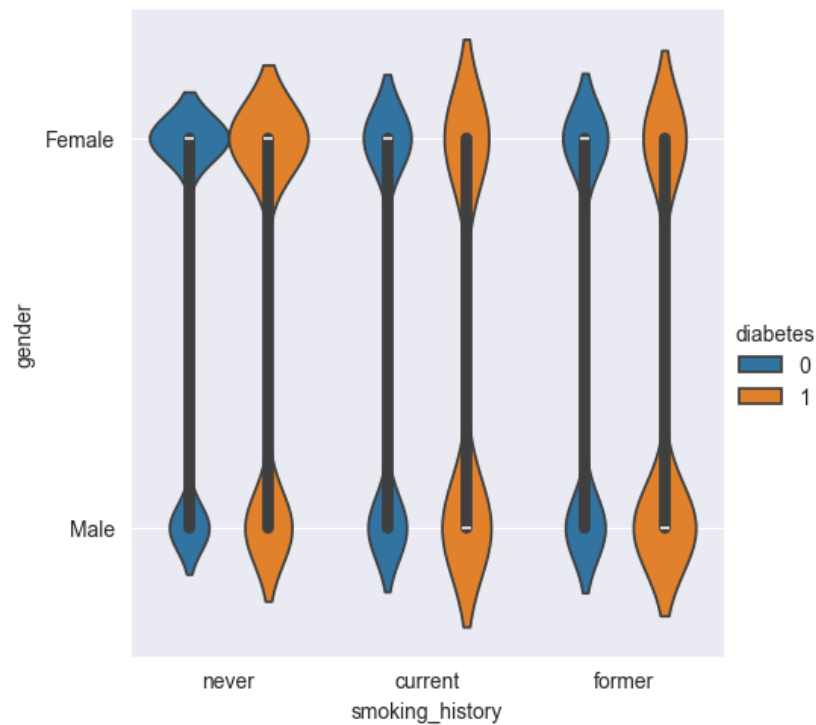


Figura 2.4: Grafico che mostra come i casi di diabete si distribuiscono in relazione al sesso e allo stato di fumatore

Nel caso maschile, per i non fumatori i casi di diabete si distribuiscono equamente, con una leggera crescita dei casi di diabete per i fumatori. C'è da dire, però, che il grafico prende in esame relazioni binarie e non lo stato medico di un paziente, che quindi è più complesso e potrebbe non dipendere da queste relazioni. Un modello che si basi fortemente su queste relazioni, risulterebbe inefficiente.

Con l'aiuto del grafico mostrato in **Figura 2.5**, possiamo esaminare l'andamento della distribuzione dei casi di diabete per fasce di età e sesso. I casi di diabete si distribuiscono su fasce di età più alte, equamente per i due generi, in contrasto con ciò che avremmo potuto trarre dal grafico precedente. Questo grafico ci aiuta a rafforzare l'idea che le persone di fasce di età avanzate sono più soggette al diabete e quindi la variabile **Age** possa essere discriminata per la predizione dei casi di diabete.

Si decide, quindi, di mantenere le features già presenti, per i motivi sopra elencati accompagnati dalle performance riscontrate dai modelli, che saranno discusse singolarmente modello per modello nella sezione seguente.

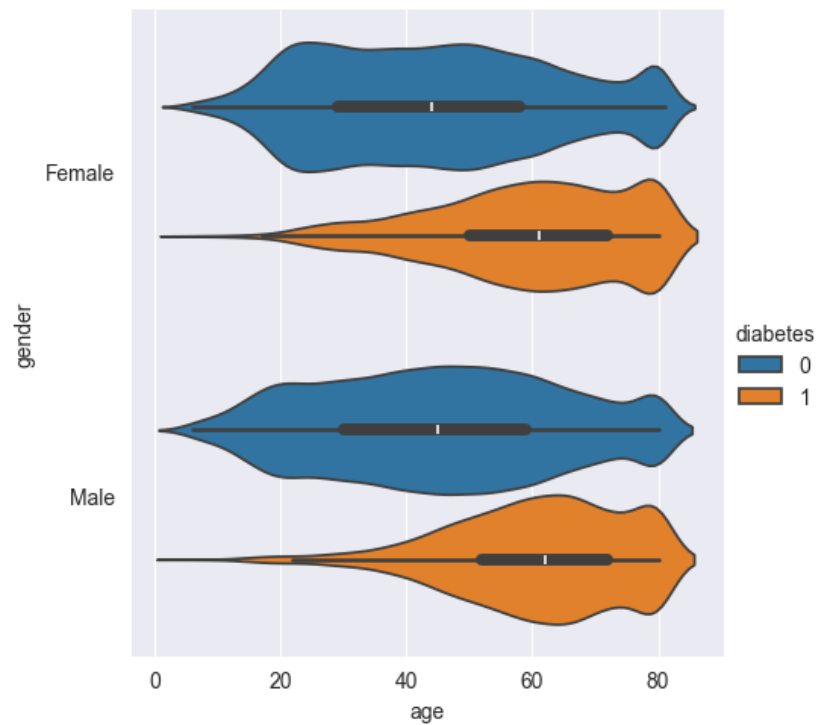


Figura 2.5: Distribuzione dei casi di diabete per fasce di età e distinte per sesso

2.3 Model Evaluation

2.3.1 Metriche di performance usate

Prima di poter discutere, analizzare e valutare i modelli scelti, è importante definire quali siano le metriche adottate per la valutazione delle performance.

Da prima, è opportuno definire i concetti fondamentali sul quale si basano le metriche:

- **Veri positivi TP:** Le classi correttamente predette come positive.
- **Falsi positivi FP:** Le classi scorrettamente predette come positive.
- **Veri negativi TN:** osservazioni negative classificate correttamente.
- **Falsi negativi FN:** osservazioni positive classificate erroneamente come negative

Avendo definito questi concetti base, elenchiamo le metriche scelte:

- **Accuracy:** L'accuratezza è una metrica che misura la frequenza con cui un modello predice correttamente la variabile dipendente.

- **Precision:** La precisione è il rapporto $tp/(tp + fp)$, dove tp è il numero di veri positivi e fp il numero di falsi positivi. La precisione è intuitivamente la capacità del classificatore di non etichettare come positivo un campione che è negativo.
- **Recall:** La **recall** è il rapporto $tp/(tp + fn)$, dove tp è il numero di veri positivi e fn il numero di falsi negativi. La **recall** è la capacità del modello di classificare correttamente tutti i campioni positivi.
- **F1-score:** La **F1-score** può essere interpretata come una media armonica tra **precision** e **recall**, dove un valore è ottimale tanto più si avvicini ad 1, nel caso opposto 0. La formula per calcolare la **F1-score** è:

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

- **Area sotto la curva ROC (auc_roc):** La curva **ROC** rappresenta graficamente il tasso di veri positivi (TPR) in rapporto al tasso di falsi positivi (FPR) a diverse soglie di classificazione. Il punteggio **auc_roc** indica l'efficacia del classificatore nel distinguere le classi positive da quelle negative. Può assumere valori da 0 a 1.

Un valore **auc_roc** più elevato indica prestazioni migliori. Un modello perfetto avrebbe un'AUC di 1, mentre un modello casuale avrebbe un'AUC di 0,5.

- **Average Precision score:** Che noi chiameremo **average_auc**, è una metrica che riassume in un unico valore l'andamento della curva Precisione-Recall di un classificatore. In pratica, l'AP corrisponde alla media pesata delle precisioni ottenute ai vari livelli di recall. L'Average Precision si può interpretare come l'area sotto la curva precisione-recall (AUC-PR) calcolata con un metodo a "somma pesata" piuttosto che con interpolazione lineare. In formula:

$$AP = \sum_n (R_n - R_{n-1}) P_n$$

dove P_n e R_n sono precisione e recall al n-esimo punto operativo (a un certo threshold) e $R_0 = 0$ per convenzione.

2.3.2 Strategie di validazione

Per valutare il nostro set di modelli, si è deciso di dividere il test dei modelli in due fasi:

- **Prima fase:** Durante la fase di *fine-tuning* dei modelli, si opta per di dividere il dataset secondo il seguente rapporto: 80% training e 20% test.
- **Seconda fase:** Una volta scelti gli iperparametri migliori, si verifica la robustezza del modello usando **k-fold-cross-validation**, con $k = 10$.

Così facendo nella prima fase possiamo esplorare uno spazio più ampio di parametri usando minori risorse e successivamente testare la soluzione migliore usando dei metodi più efficaci.

2.4 Machine learner adottati

Per raggiungere i nostri obiettivi di ricerca, non è solo importante il dataset. Una parte cruciale riguarda la scelta dei modelli, che saranno questi ultimi a sfruttare i nostri dati per poter predire correttamente il diabete.

Visti gli **Obiettivi di ricerca 2.1**, si sceglie di selezionare dei modelli dissimili fra loro o per la maggior parte, così da poter avere responsi diversi. La scelta è ricaduta sui seguenti modelli:

- i **Random forest**
- ii **k-nearest neighbors**
- iii **AdaBoostClassifier**
- iv **Logistic regression**
- v **MLP**
- vi **XGBoost**

Andiamo nel dettaglio dei modelli elencati, fornendo una panoramica dei modelli, le scelte degli iperparametri e le performance ottenute.

2.4.1 Random forest

Un random forest classifier è un meta-estimatore composto da vari alberi decisionali che si concentrano su sotto-campioni e usa la media per migliorare l'accuratezza predittiva e controllare l'overfitting. " Random forest è un algoritmo di apprendimento automatico di uso comune, registrato da Leo Breiman e Adele Cutler, che combina l'output di più decision trees per raggiungere un unico risultato. La sua facilità d'uso e la sua flessibilità ne hanno favorito l'adozione, in quanto gestisce sia i problemi di classificazione che quelli di regressione " [4].

Vantaggi

- **Robustezza all'overfitting** Grazie all'ensemble di molti alberi indipendenti, il modello media le predizioni, riducendo il rischio di overfitting rispetto a un singolo albero decisionale.
- **Resistenza al rumore e outlier** Il comportamento a "voto di maggioranza" degli alberi diminuisce l'effetto di esempi rumorosi o outlier sul risultato complessivo.
- **Stima dell'importanza delle feature** Fornisce ranking delle variabili più informative sulla classificazione, aiutando nell'interpretazione e nella selezione di feature.

Svantaggi

- **Costo computazionale** Tempo di addestramento e predizione possono crescere rapidamente con l'aumento del numero di alberi o della profondità massima di ognuno, richiedendo più memoria e potenza di calcolo.
- **Tendenza a bias** In presenza di classi molto sbilanciate, tende a prediligere la classe più frequente a meno di interventi per mitigare ciò.
- **Scarsa extrapolazione** Come tutti gli alberi decisionali, non riesce a generalizzare oltre il range dei dati visti (es. feature al di fuori dei valori di training).

2.4.2 XGboost

XGBoost (eXtreme Gradient Boosting) è una libreria di apprendimento automatico distribuita e open source che utilizza alberi decisionali potenziati dal gradiente, un algoritmo di potenziamento dell'apprendimento supervisionato che utilizza la discesa del gradiente. È noto per la sua velocità, efficienza e capacità di scalare in modo efficace con set di dati di grandi dimensioni [5].

Vantaggi

- **Elevata accuratezza e performance** Utilizza boosting gradientale con ottimizzazioni a livello di sistema, che ne fanno uno degli algoritmi più rapidi su dataset strutturati.
- **Controlli di overfitting integrati** Offre regolarizzazione L1 e L2 sui pesi, insieme a parametri come **max_depth**, **min_child_weight** e **gamma**, che aiutano a prevenire l'overfitting.
- **Gestione automatica dei valori mancanti** Impara dinamicamente la direzione migliore (left/right) per gli split quando incontra missing values, senza doverli imputare manualmente.

Svantaggi

- **Complesso tuning degli iper-parametri** Presenta molteplici iperparametri, complicando il corretto tuning del modello.
- **Consumo di memoria e risorse** Soprattutto su dataset molto grandi o con molte features, l'allocazione degli alberi e delle strutture temporanee può richiedere RAM importante.
- **Curva di apprendimento** Richiede una buona familiarità con il paradigma del boosting e con l'impatto di ciascun iper-parametro; per un principiante può risultare più ostico di altri modelli ensemble come Random Forest.

2.4.3 MLP

Il multi-layer perceptron (MLP) è un algoritmo di apprendimento supervisionato che costruisce una certa funzione $f : R^m \rightarrow R^o$ costruita sui dati appresi dal dataset durante l'allenamento, dove m è il numero della dimensione dell'input, in questo caso è il numero di features e o è la dimensione dell'output, la nostra variabile da predire [6].

Vantaggi

- **Capacità di apprendere relazioni non lineari** È in grado di approssimare qualsiasi funzione continua
- **Flessibilità architetturale** Tramite il numero di nodi interni e i molteplici iperparametri, possiamo adattare al meglio il modello per il problema specifico.
- **Generalizzazione** Con una corretta regolarizzazione e sufficiente quantità di dati, un MLP ben configurato tende a generalizzare meglio rispetto a modelli lineari o shallow.

Svantaggi

- **Rischio di overfitting** Reti neurali troppo grandi posso inficiare sulle prestazioni del modello, portandolo ad un adattamento estremo ai dati di training.
- **Scelta e ottimizzazione degli iper-parametri** Numero di strati, numero di neuroni, learning rate, tipo di ottimizzatore, funzioni di attivazione, batch size: ognuno influisce sulle prestazioni e richiede tuning, il che aggiunge complessità al modello.
- **Costi computazionali elevati** Addestramento e predizioni su reti di media o grande taglia richiedono più risorse (GPU/CPU, memoria) e tempi più lunghi rispetto a modelli più semplici.
- **Interpretabilità limitata** Una rete con molteplici nodi interni, rende difficile capire il perché delle scelte effettuate dal modello.

- **Sensibilità alla normalizzazione dei dati** Richiede dati normalizzati, altrimenti le performance del modello potrebbero risentirne.

2.4.4 k-nearest neighbors

L'algoritmo di apprendimento automatico **k-nearest neighbors (KNN)** è un algoritmo di apprendimento supervisionato non parametrico tra i più semplici e allo stesso tempo popolari nell'apprendimento automatizzato, che può essere usato sia per problemi di regressione che di classificazione. **KNN** si basa sull'idea che elementi simili si trovano vicini tra loro nello spazio delle feature. Quando si deve classificare un nuovo campione, l'algoritmo cerca i k punti più vicini nel dataset di training e assegna la classe in base alla maggioranza (nel caso della classificazione) oppure calcola una media (nel caso della regressione).

Per determinare i K punti più vicini a un determinato punto di interrogazione, è necessario calcolare la distanza tra il punto di interrogazione e gli altri punti di dati. Queste metriche di distanza consentono di formare i limiti decisionali, che partizionano i punti di query in aree diverse. I confini decisionali vengono comunemente visualizzati con i diagrammi di Voronoi. [7] Le metriche più comunemente adottate si basano sulla formula generale della **Distanza di Minkowski**.

$$\sum_{i=1}^n |y_i - x_i|^{1/p}$$

In contesti con vettori a numeri reali viene usata la **distanza euclidea**

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

L'altro metodo molto diffuso che può essere ricavato dalla formula generale, è la **distanza di Manhattan**, che misura il valore assoluto fra due punti e viene visualizzata come una griglia. La formula è la seguente:

$$\sum_{i=1}^n |y_i - x_i|$$

Vantaggi

- **Facile da imparare:** KNN è uno dei modelli di ML più facili da capire e comprendere, avendo anche ottime prestazioni, ed si implementa con facilità tramite sklearn.

- **Adattabilità:** l'algoritmo si adatta molto velocemente ai cambiamenti apportati da nuovi dati.
- **Pochi iperparametri:** il KNN richiede solo un valore k e una metrica di distanza, molto meno rispetto a molti altri modelli di apprendimento automatico.

Svantaggi

- **Scalabilità:** Siccome KNN salva in memoria i dati con cui si è addestrato, per modelli grandi non è efficiente in termini computazionali sia di spazio che di tempo.
- **Riduzione della dimensionalità:** Sensibilità a dati rumorosi e a feature non normalizzate: è importante pre-processare i dati per garantire che tutte le feature abbiano un peso simile nel calcolo della distanza.
- **Incline all'overfitting:** A causa della "maledizione della dimensionalità", il KNN è anche più incline all'overfitting. Anche se la feature selection e lo scaling dei dati aiutino ad attenuare questo problema, anche il valore di k può influire sul comportamento del modello. I valori più bassi di k possono adattarsi eccessivamente ai dati, mentre i valori più alti di k tendono a sotto-adattarsi ai dati.

2.4.5 AdaBoostClassifier

AdaBoostClassifier (ADA) è un modello di machine learning basato su un insieme di decision tree pesati poco profondi.

ADA inizia con un solo semplice decision tree e ad ogni errore di classificazione costruisce nuovi decision tree per correggere gli errori e migliorarsi.

Ogni decision tree è addestrato sull'intero dataset, ma di volta in volta viene adattato secondo campioni pesati basati sulle precedenti misclassificazioni. ADA classifier, essendo un classificatore, per decidere e classificare un'istanza del problema, fa uso di un sistema di voti pesati, dove i decision tree migliori hanno maggior rilevanza [8].

Vantaggi

- **Non richiede tuning eccessivo.** Ha pochi iper-parametri (numero di stumps, learning rate), ed è relativamente stabile al variare del learning rate.
- **Margine di classificazione.** A livello Teorico massimizza il margine fra classi, contribuendo a ridurre l'errore di generalizzazione.
- **Focalizzazione sui casi difficili.** Ad ogni iterazione, aumenta il peso degli esempi mal classificati, costringendo i successivi learner a concentrarsi sui pattern più complessi o rari.

Svantaggi

- **Sensibilità al rumore e agli outlier.** Poiché dà peso crescente agli esempi mal classificati, rumore e outlier possono ricevere pesi molto elevati e “disturbare” l'intero modello, degradandone la performance.
- **Overfitting in casi estremi.** Sebbene in molti casi sia robusto all'overfitting, con dataset molto rumorosi o con troppi boosting round può iniziare a modellare il rumore.
- **Non ideale per dataset sbilanciati.** Se le classi sono fortemente sbilanciate, gli esempi minoritari possono essere trattati come “difficili” e ricevere troppo peso, portando a instabilità.
- **Limitato binario.** Esistono estensioni multi-classe (SAMME, SAMME.R), ma il meccanismo originale è pensato per due sole etichette.
- **Costo computazionale crescente** Ad ogni iterazione bisogna ricalcolare pesi ed eseguire un nuovo weak learner: con molti round e dataset grandi, i tempi possono diventare rilevanti.

2.4.6 Logistic regression

Il Logistic Regression Classifier è un modello di apprendimento supervisionato che, a partire da un insieme di variabili di input, calcola la probabilità che un'osservazione appartenga a una certa classe.

Durante l'addestramento, il modello regola i pesi ω e il bias b per massimizzare la coerenza tra queste probabilità stimate e le etichette reali, minimizzando l'errore di log-verosimiglianza. Alla fine, per decidere la classe definitiva, si fissa una soglia (di solito 0.5): se $\sigma(z) \geq 0.5$ classifica come positivo, altrimenti come negativo [9].

Vantaggi

- **Efficienza computazionale:** Il modello rimane efficiente in termini di complessità algoritmica anche su dataset grandi.
- **Regolarizzazione semplice:** L1/L2 su pesi aiuta a prevenire overfitting e, con L1, induce selezione automatica delle feature.
- **Interpretabilità:** i coefficienti ω si possono leggere come variazioni log-odds, quindi è chiaro come ogni feature influisce sulla probabilità di classe.
-

Svantaggi

- **Relazione lineare:** non cattura nativamente interazioni o dipendenze non lineari.
- **Sensibilità a outlier:** valori estremi o feature altamente correlate possono distorcere la frontiera decisionale. **Gestione classi sbilanciate:** senza tecniche di bilanciamento o pesi inversi, tende a favorire la classe dominante.

2.5 Tecniche di explainability

Le metodologie usate per giungere al conseguimento dei target di explainability relativi ai machine learners impiegati, sono i framework **SHAP**(SHapley Additive exPlanations) e **LIME**(Local Interpretable Model-agnostic Explanations).

Come anticipato nelle sezioni precedenti, tali metodologie saranno analizzate nel dettaglio nel seguito.

2.5.1 LIME

Lime è una tecnica di AI explainability che spiega il funzionamento di ogni modello poiché è model-agnostic, andando a creare dei modelli surrogati a livello della predizione locale, cioè, esso si occupa di spiegare il comportamento del modello nella singola predizione e non come esso agisce [10]. Lime prende in esame il modello *black box* in esame, senza prendere in considerazione i dati di test; quando al modello è richiesta una predizione, ecco che subentra LIME. Analizza come il modello si comporta al variare dei dati dati in pasto al modello in un certo intorno, una volta fatto ciò crea un nuovo dataset a partire dalle permutazioni dei campioni usati in precedenza e sulla predizione del modello *black box*, così da poter costruire un modello interpretabile sulla predizione locale [11].

2.5.2 SHAP

SHAP (Shapley Additive explanations) è un approccio che fa uso della teoria dei giochi per poter spiegare l'output di qualsiasi modello di machine learning. SHAP si basa sui valori di Shapley, un concetto della teoria dei giochi cooperativi, che assegna a ciascun giocatore - in questo caso, ogni feature - un valore di contributo "equo" alla riuscita di una coalizione (la previsione del modello).

Per ogni predizione, Shap confronta la risposta del modello in assenza di una determinata feature con la predizione ottenuta includendola, tenendo conto di tutte le possibili permutazioni. Una volta calcolati i valori Shapley, il valore della predizione sarà dato dal valore atteso $E(x)$ più la somma di tutti i valori Shapley ottenuti.

Le soluzioni che questo modello di explainability di modelli AI si basano su tre concetti chiave [12].

- **local accuracy:** come detto in precedenza.

Il valore della predizione sarà data dal valore atteso $E(x)$ più la somma di tutti i valori Shapley ottenuti.

- **Missingness:** Le feature che non sono prese in esame nella predizione, non hanno impatto sul modello e quindi il suo valore shap sarà 0.
- **Consistency:** I valori SHAP non cambiano quando cambia il modello, a meno che non cambi il contributo di una caratteristica.

Questo approccio consente di avere delle spiegazioni molto dettagliate, che fanno uso di concetti matematici basati sulla teoria dei giochi, potendo analizzare un modello senza essere influenzati dal dataset mediante l'uso di grafici semplici ed intuitivi, presenti nel framework di Shap.

3.1 Random Forest

3.1.1 Iperparametri

Per affrontare al meglio la parametrizzazione dei modelli, in questo caso di random forest, dobbiamo comprendere quali siano i parametri principali:

- **n-estimators:** numero di alberi nella foresta.
- **max-depth:** profondità massima di ciascun albero.
- **min-samples-split:** numero minimo di campioni richiesti per suddividere un nodo interno.
- **min-samples-leaf:** numero minimo di campioni in un nodo foglia.
- **max-features:** numero di caratteristiche da considerare per il miglior split.
- **bootstrap:** se effettuare il campionamento con ripetizione (True/False).

Nel caso del modello preso in analisi, tra quelli principali sopraelencati, sono stati scelti **max-depth**, **n-estimators**, **bootstrap**. **Bootstrap** non ha avuto una grande rilevanza, per quanto riguarda il numero di alberi, riportiamo attraverso la **Tabella 3.1**

come il modello cambi al variare del numero di alberi e perché abbiamo considerato come valore ottimale, il valore di default di 100.

n-alberi	accuracy	f1-score	precision	auc-recall
50	95.78%	95.51%	91.06%	86.38%
70	95.68%	95.41%	89.94%	86.49%
100 default	95.67%	95.39%	90.20%	86.60%
130	95.71%	95.44%	90.62%	86.71%
200	95.71%	95.44%	90.62%	86.70%

Tabella 3.1: Prestazioni di random forest al variare del numero di alberi

Ricordiamo che in questo studio, sono sì importanti tutte le metriche, ma siccome ci interessa predire correttamente la classe dei diabetici, il peso maggiore lo si dà alla **Recall** e alla **f1-score**. Avendo fatto queste assunzioni, il valore di base ha raggiunto valori migliori.

Considerando ora come cambia il modello al cambiare dei valori **max-depth** (Figura 3.2) , consideriamo in questo caso come valore migliore 10. Si vuole sottolineare come, in generale, questo modello con i parametri scelti abbia performance che si discostano di valori di un intorno dell' 1%, quindi uno di questi parametri mostrati si potrebbe considerare come una buona scelta.

max-depth	accuracy	f1-score	precision	auc-recall
5	95.98%	95.58%	100%	85.33%
10	96.06%	95.67%	99.74%	87.89%
15	95.97%	95.64%	95.38%	87.82%
21	95.76%	95.48%	91.32%	86.99%
30	95.70%	95.42%	90.42%	86.60%

Tabella 3.2: Come si comporta random forest aumentando la lunghezza massima che può raggiungere

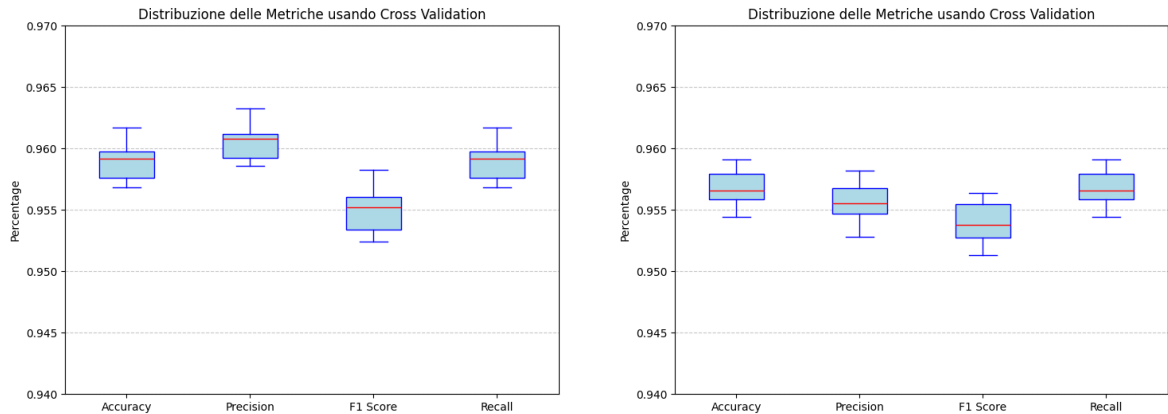


Figura 3.1: Confronto delle prestazioni del modello con parametri di default sulla destra e usando max-depth 10 sulla sinistra usando la K-cross-validation

Avendo trovato i parametri che ci soddisfano di più, andremo ad applicare un **K-cross-validation**. Come mostra la Figura 3.1, vi è un leggero incremento delle prestazioni del modello con il valore **max-depth** di 10, in contrasto a quanto si evince dal modello con parametri di default. La validità di queste performance è confermata dalla **K-cross-validation** con $k = 10$ che ci aiuta a capire se il modello soffra di overfitting. Nel corso di questa ricerca, quando si parlerà di **K-cross-validation** si fa riferimento sempre ad uso scaglionamento in 10 iterazioni.

3.1.2 Explainability

Lime

Mediante l'ausilio di Lime si è analizzata una predizione locale, così da poter capire quali siano state le scelte che hanno portato il modello verso la classe positiva o negativa. La **Figura 3.2**, mostra come la variabile **HbA1c** sia la più influente, dando un contributo maggiore verso la classe negativa, viceversa con impatto minore per **glucose**.

Da quanto esaminato, possiamo dire che il modello prenda in esame le giuste features, andando a considerare le giuste relazioni.

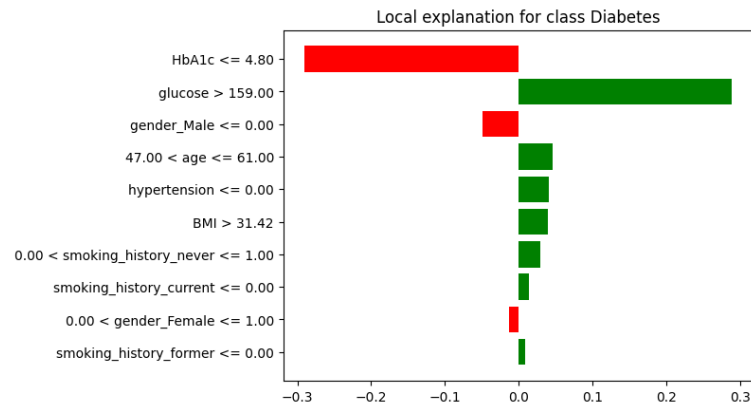


Figura 3.2: Grafico lime che mostra una predizione etichettata come non diabetico

Shap

Dapprima si è valutato il modello su istanze di **falsi negativi**, come riportato in **Figura 3.3**, dove il modello dà una maggior importanza alla variabile **age: 77** e **BMI: 21.21**, dove quest'ultimo ha spinto maggiormente il modello verso la classe negativa, portando all'errore. In aggiunta vi è una poca rilevanza data alle features chiave, aventi valori **HbA1c: 6.6** e **glucose: 130**, siccome questi ultimi sono fuori dai valori normali dovrebbero portare ad una predizione verso la classe positiva, diversamente da quanto successo.

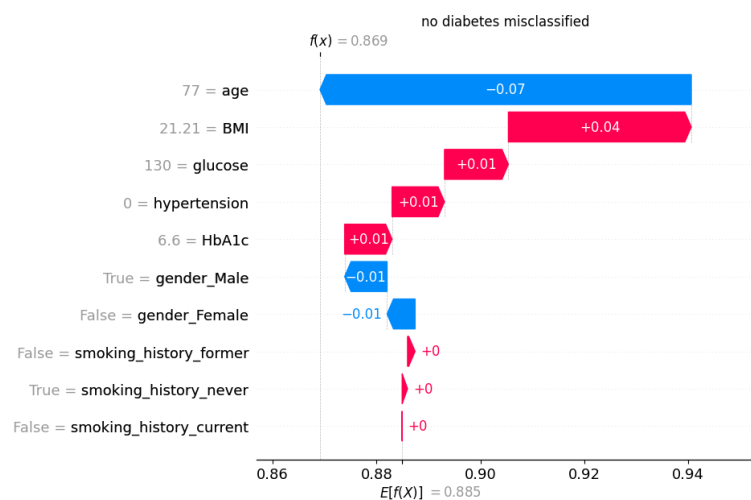


Figura 3.3: Grafico waterfall mostrante un falso negativo di random forest

Passando ad analizzare un'istanza correttamente classificata come positiva, invece, come riportato in **Figura 3.4** che il modello abbia preso in considerazione solamente

la variabile **glucose**, con un valore altissimo di 260, non può che ricadere nella classe positiva.

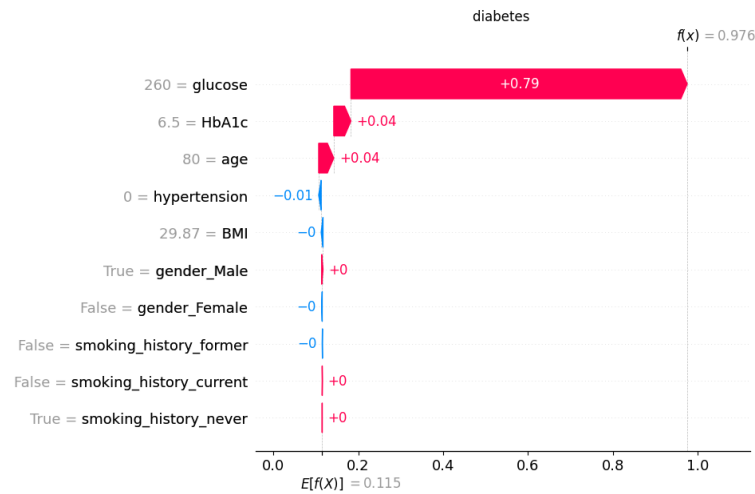


Figura 3.4: Grafico waterfall mostrante un vero positivo di random forest

Si è poi usata una heatmap di shap, ordinata secondo i valori di $f(x)$, così da poter analizzare il comportamento globale del modello.

Dalla **Figura 3.5** si può osservare come il modello dia un'importanza quasi nulla al **gender**, **smoking history**, dando maggior importanza alle features chiave, seguite da **age** e **BMI**. Si osserva anche come, per valori crescenti di $f(x)$ ci sia una marcata variazione del contributo dalla classe negativa a quella positiva per le feature chiave. In linea generale, il modello non si è comportato male, andando ad agire secondo una "logica" razionale nelle sue scelte.

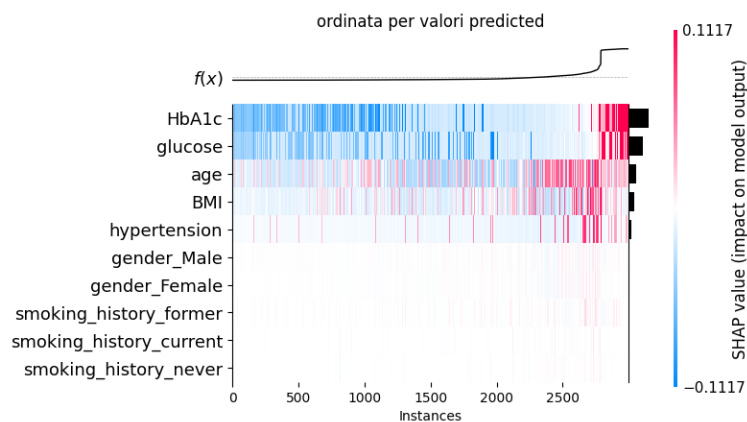


Figura 3.5: Heatmap shap rappresentante il comportamento del modello random forest

3.2 XGBoost

3.2.1 Iperparametri

Sono stati scelti tre parametri chiave per il **fine-tuning** del modello: **learning rate**, **n_estimators** e **max_depth**. Abbiamo testato il modello con le possibili combinazioni della terna di queste variabili, mostriamo attraverso la **Figura 3.6**.

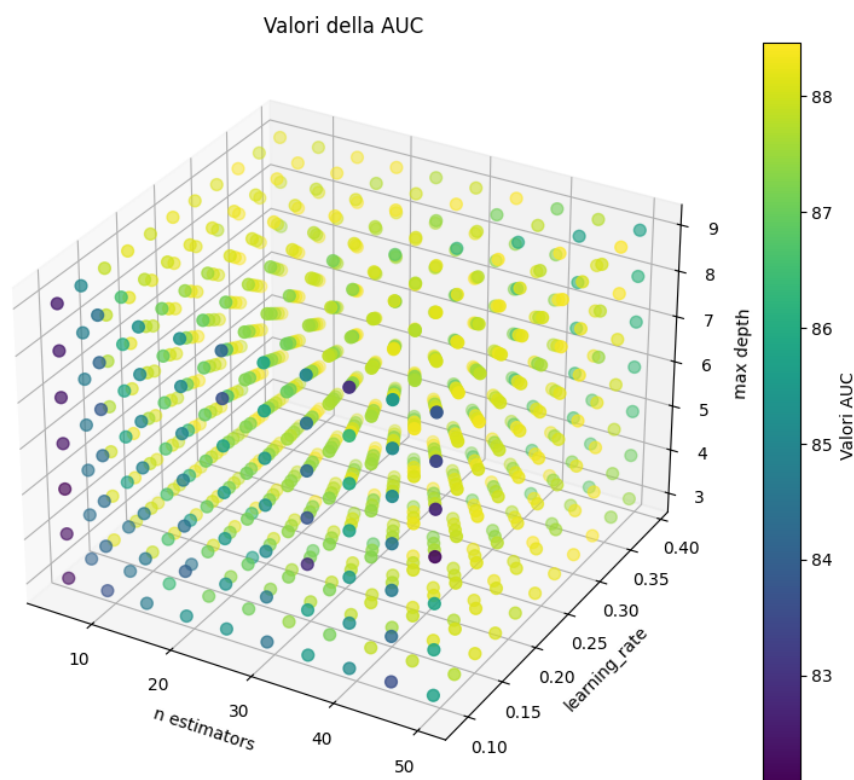


Figura 3.6: Grafico che mostra come cambia il valore auc-roc rappresentando valori più alti in gialli e valori inferiori tendenti al viola

Il grafico ci permette di mettere in relazione le variabili indipendenti con la variabile dipendente che vogliamo mostrare, in questo caso, i valori **AUC**, usando come *4d* il colore. Avendo osservato il grafico, ricaviamo che un valore troppo basso o troppo alto di **learning rate** infici molto le performance del modello, un discorso analogo vale anche per **max_depth**, dove il valore ottimale di questa variabile si attesta tra valori compresi tra 3 e 5.

Il contributo della variabile **n_estimators** sembra marginale in questo contesto, non pesando eccessivamente sul modello; In questo caso potrebbe essere dato da un

dataset con una complessità limitata.

Data le conclusioni di cui sopra, scegliamo di testare il nostro modello con i parametri seguenti: **n_estimators = 50**, **learning rate= 0.27** e **max_depth = 4**. Il modello si

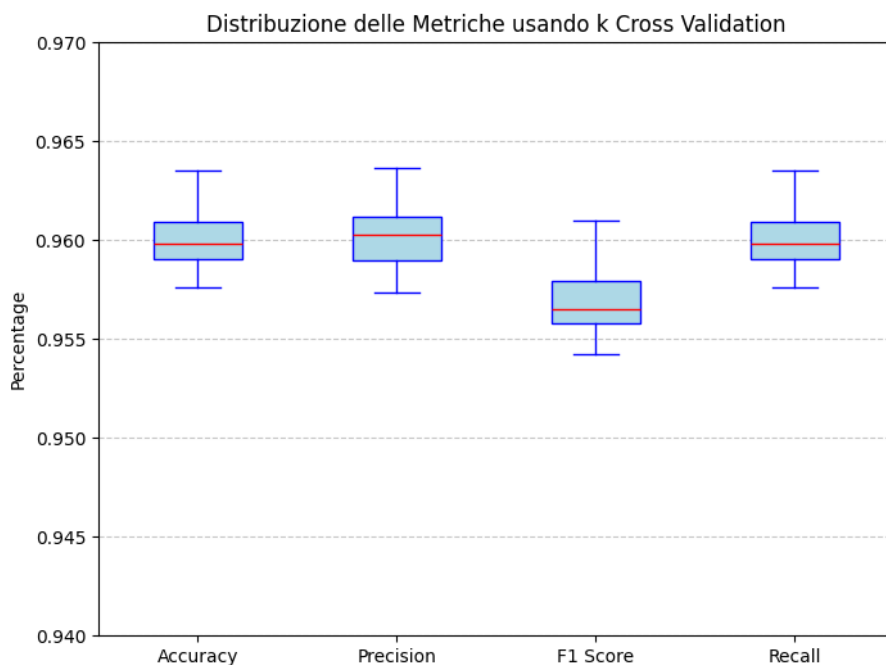


Figura 3.7: Distribuzione delle metriche di xgboost usando k-cross-validation

comporta adeguatamente con gli iperparametri scelti, come viene illustrato in **Figura 3.7**. Si può osservare come le metriche scelte non mostrano valori fuori dalla norma, e sono tutte molto stabile in un dato range di valori.

3.2.2 Explainability

Tramite la **Figura 3.8**, si può osservare come il modello per classificare l'istanza come positiva abbia dato un peso maggiore alle feature chiave rispetto alle altre, poiché valori superiori a 6.20 per **HbA1c** e di 159 per **glucose** siano determinanti nella valutazione della scelta fatta, mostrando effettivamente una buona "logica" dietro le scelte del modello.

Lime

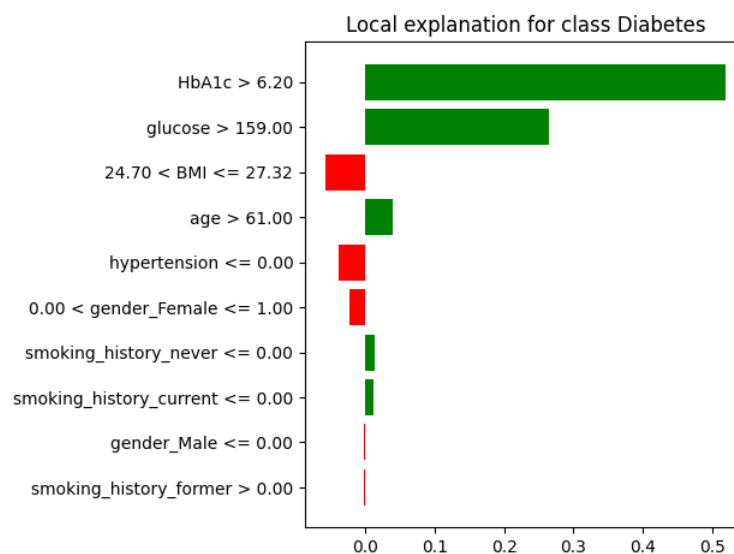


Figura 3.8: grafico Lime per l'interpretazione delle scelte che hanno portato a classificare l'istanza corrente come positiva

Shap

Utilizzando un grafico waterfall di shap, **Figura 3.9**, abbiamo visualizzato le scelte fatte dal modello che lo hanno indotto in un falso negativo.

Si può osservare come il modello abbia dato un'importanza maggiore ad **age**, **BMI**, piuttosto che alle features chiave, così come fatto per il modello precedente, portandolo di fatto a classificare l'istanza in modo errato. Va notato come, nei casi di valori limite delle features chiave, il modello tenda a sbagliare e dare meno importanza ad esse.

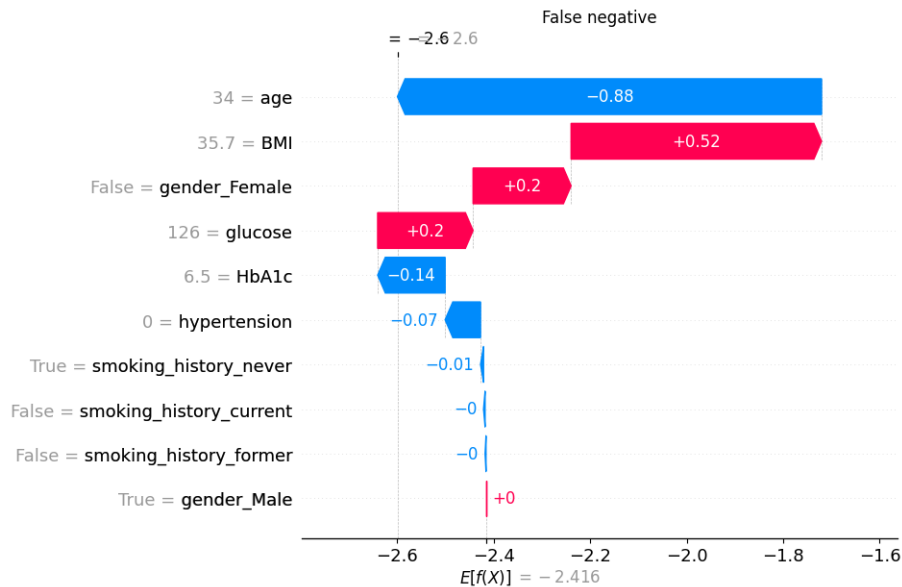


Figura 3.9: Grafico waterfall mostrante un falso negativo di Xgboost

Un caso diverso, invece, per quanto riguarda una classificazione corretta di classe positiva, dove viene preso in esame semplicemente il valore della **HbA1c**, come evidenziato dal grafico in **Figura 3.10**. Con valori poco dissimili di **HbA1c** e **glucose**, il modello si è comportato in maniera differente, avendo un contributo verso la classe positiva da parte di **HbA1c** di +8.27, molto diverso dal caso preso in esame precedentemente; questo ci evidenzia come il modello possa fare fatica su certi casi limite a predire correttamente la classe dei diabetici.

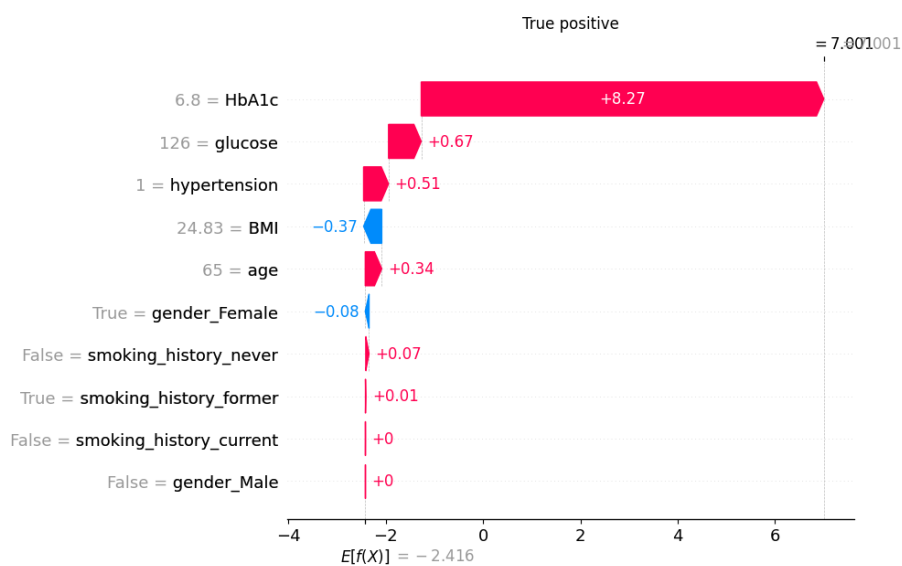


Figura 3.10: Grafico waterfall mostrante un vero positivo di Xgboost

Si discute di seguito il comportamento del modello a livello globale attraverso una heatmap, illustrata in **Figura 3.11**. Come per **Random Forest**, anche in **Xgboost** le features: **hypertension**, **gender** e **smoking history** hanno una valenza quasi nulla nelle scelte del modello. Le features elencate per importanza si sono rivelate essere quelle che ci aspetteremmo di trovare per un buon modello, rafforzando l'idea che il modello abbia una base ottima per il problema in questione.

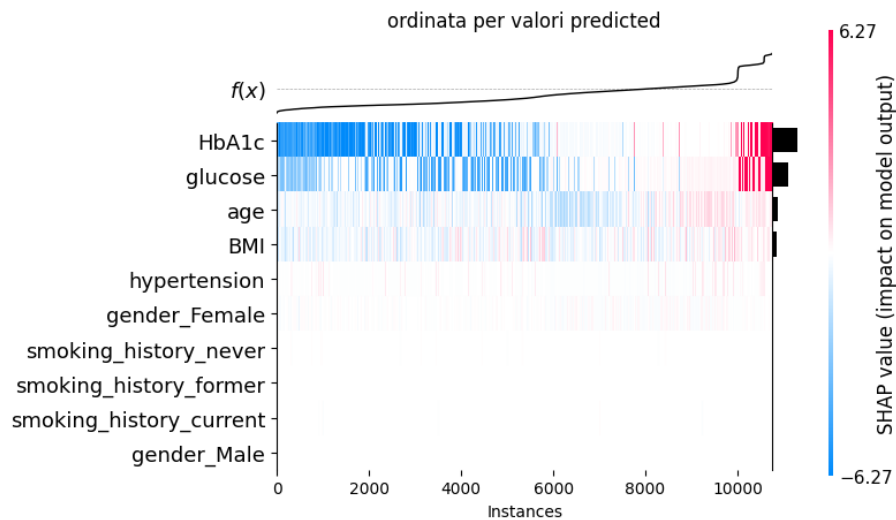


Figura 3.11: Heatmap shap rappresentante il comportamento del modello Xgboost

3.3 MLP

3.3.1 Iperparametri

Il **Multi-layer percetor** presenta molti iperparametri, tra cui **solver**, **hidden_layers**, **activation**, **alpha**, **learning rate**. Alcuni degli iperparametri possono essere cambiati solo in combinazione ad altri. In aggiunta, presenta numerosi parametri, richiedendo una conoscenza approfondita per ottenere il massimo dal modello. Data la complessità, ci limiteremo a fare *fine-tuning* sul numero di **hidden-layers**, **solver** e **learning-rate-init**

neuroni	accuracy	f1-score	precision	auc-recall
(5,2)	95.60%	95.20%	95.59%	84.18%
(15,9)	96.09%	95.76%	96.12%	87.40%
(32, 16)	96.04%	95.72%	96.04%	87.44%
(40, 20)	96.08%	95.77%	96.06%	87.51%
(64, 32)	95.89%	95.58%	95.83%	86.82%
(80, 50)	95.99%	95.67%	95.99%	87.17%

Tabella 3.3: Tabella che riporta le performance del modello al variare del numero di nodi interni della rete neurale

Dapprima abbiamo analizzato semplicemente alcune combinazioni di **hidden-layers**, come visibile in **Figura 3.3**, e considerando come parametri migliori le coppie (40, 20) e (80, 50). Questa prima analisi si concentra su un solo iperparametro per volta. Successivamente, abbiamo utilizzato un **RandomSearchcv** con una **k-fold-cross-validation** per esaminare un campione più ampio di parametri, il che ci permette tramite le suddette funzioni di scegliere i parametri su cui vogliamo testare l'algoritmo e il range di valori associati ad essi, e ci restituirà i parametri migliori secondo la misura di performance scelta, in tal caso la scelta è su **roc_auc_ovr**. Date le ottime performance dei campioni riportati in tabella, allargheremo leggermente il campo andando a considerare i parametri di cui sopra, per non sfociare in un costo troppo oneroso in termini di risorse.

```

1 pipe = Pipeline([
2     ('scaler', StandardScaler()),
3     ('mlp', MLPClassifier(max_iter=1000, early_stopping=True,
4         ↪ random_state=42))
5 ])
6 param_distributions = {
7     'mlp__hidden_layer_sizes': [(50,), (65,), (80,), (100,),
8         ↪ (50,), (5, 2), (15, 9), (32, 16), (40, 20), (64, 32),
9         ↪ (80, 50), (100, 50), (80, 60, 20),
10        ↪ (100, 80, 30), (100, 100, 50)],
11     'mlp__activation': ['relu', 'tanh'],
12     'mlp__alpha': loguniform(1e-5, 1e-1),
13     'mlp__learning_rate_init': loguniform(1e-4, 1e-1),
14     'mlp__batch_size': [32, 64, 110, 128, 256]
15 }
16 cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
17 rnd_search = RandomizedSearchCV(pipe,
18     ↪ param_distributions=param_distributions,
19     ↪ n_iter=30, cv=cv,
20     ↪ scoring='roc_auc_ovr',
21     ↪ refit=True, n_jobs=6)
22 rnd_search.fit(X_train, y_train)

```

Source Code 3.1: Codice sorgente per la ricerca automatica dei migliori parametri di MLP in un dato intervallo

I risultati della ricerca 3.1 ci hanno indicato come migliori parametri i valori seguenti:

- **hidden_layers:** (100, 80, 30)
- **activation:** relu
- **batch_size:** 32
- **alpha:** $1.6885e - 05$
- **learning_rate_int:** 0.0044

Trovati i parametri migliori usando **random search**, li abbiamo messi a confronto attraverso una **k-cross-validation** con i parametri migliori ottenuti empiricamente, cioè, prendendo in esame il modello modificando il solo parametro dei neuroni, la coppia scelta è stata (80, 50).

Esaminando i due riscontri **Figura 3.12**, vista la maggior consistenza di performance, seppur di poco e l'assenza di outlier, si è scelto di usare il modello con i seguenti parametri:

- **Hidden layers:** (80, 50).

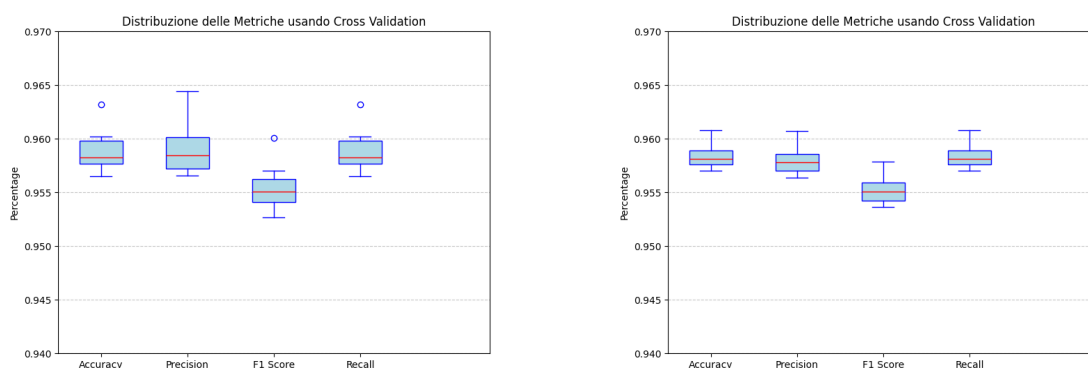


Figura 3.12: Confronto delle prestazioni del modello con parametri empirici(destra) e parametri tramite random search(sinistra) usando la K-cross-validation

3.3.2 Explainability

Lime

Il grafico Lime illustrato in **Figura 3.13**, da esso si ricava che le features ad avere il peso maggiore sono **HbA1c** e **glucose**. I risultati ottenuti evidenziano un buon comportamento del modello, dando il giusto peso alle features chiave, ponendole come le più discriminanti.

Shap

La **Figura 3.14** mostra il grafico shap waterfall di una predizione errata, in questo caso, un **falso negativo** ed il perché delle scelte che hanno portato il modello a classificare questa istanza negativa al diabete (in modo errato). Il modello ha dato maggior peso alla **BMI**, spostando la predizione verso la classe dei non diabetici, così come per il **glucosio** anche se con un impatto minore (valori di **glucosio** di 130 che non sono valori nella norma se a digiuno); invece, un valore di 6.6 per la **HbA1c** aveva indirizzato (correttamente) il modello verso la classe positiva.

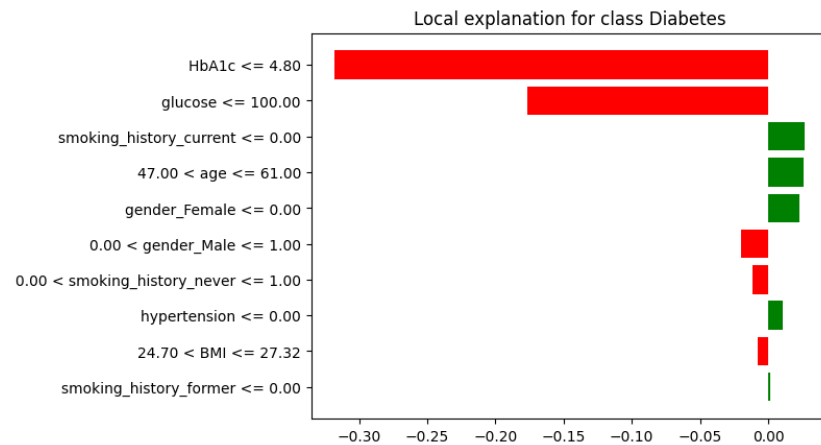


Figura 3.13: Grafico lime MLP

Dovendo valutare la suddetta istanza, il modello ha comunque preso in considerazione parametri chiave, seguendo una logica.

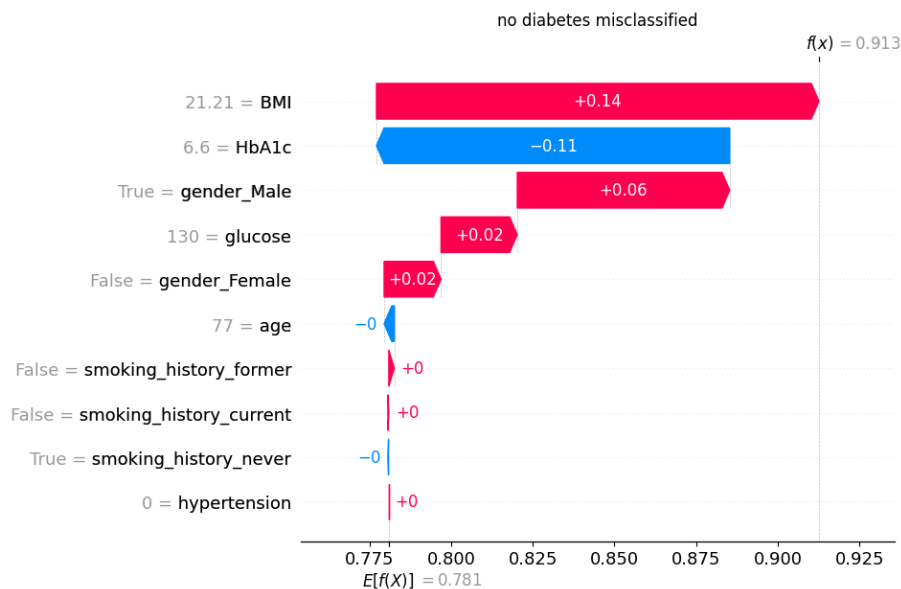


Figura 3.14: Grafico waterfall mostrante un falso negativo di MLP

Nel caso della **Figura 3.15**, ci troviamo in un'istanza correttamente classificata come positiva, prendendo in esame e dando maggiore peso alla variabile **Glucose**, dove il suo valore si attesta su 260, un chiaro indicatore che il paziente sia diabetico.

Successivamente si è analizzato il comportamento generale del modello attraverso una heatmap, come mostrato in **Figura 3.16**. Ordinato secondo i valori della funzione $f(x)$, si nota come le features con una rilevanza maggiore sono le features chiave che avevamo definito nella sezione 2.2.1, e come al crescere della funzione $f(x)$, sia

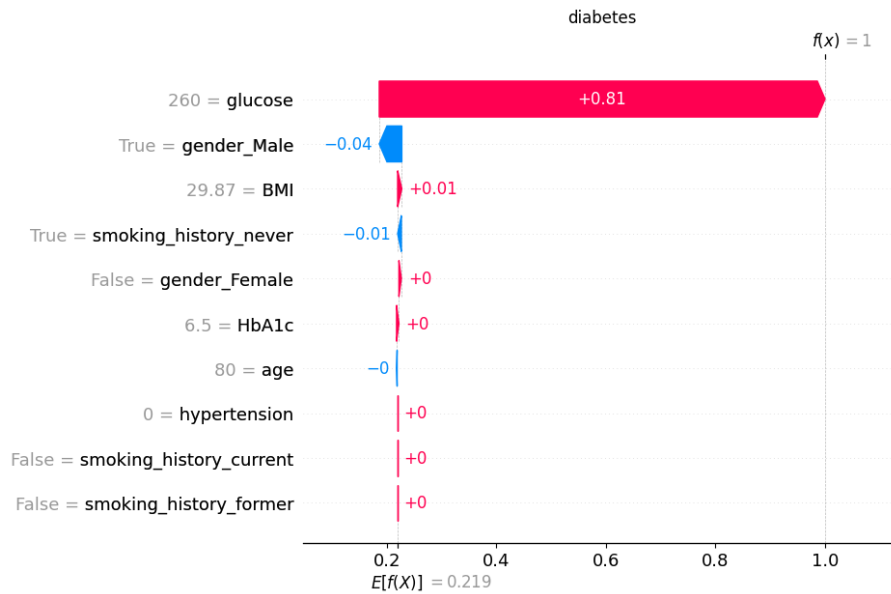


Figura 3.15: Grafico waterfall mostrando un vero positivo di MLP

particolarmente discriminante **HbA1c e glucose**, marcando una forte relazione fra queste due variabili nella distinzione delle due classi target, per quanto riguarda il modello in questione.

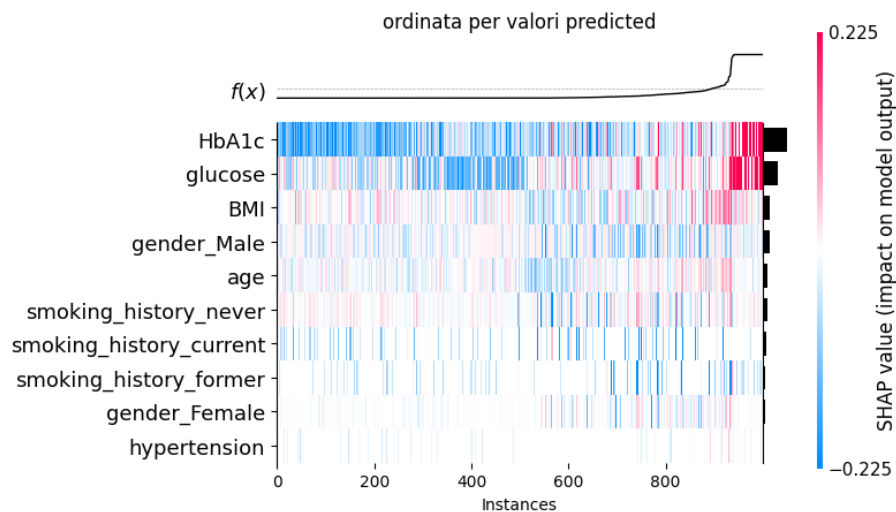


Figura 3.16: Heatmap shap rappresentante il comportamento del modello mlp

3.4 k-nearest neighbors

3.4.1 Iperparametri

Il modello kNN ha ottenuto ottime prestazioni con valori parametrici di **k-neighbors** = 13 e parallelizzazione di 4 come mostrato nella **Tabella 3.4**.

K-neighbors	accuracy	f1-score	precision	auc-recall
7	94.57%	94.05%	88.90%	80.75%
9	94.71%	94.17%	90.90%	80.96%
11	94.67%	94.09%	92.09%	81.30%
13	94.68%	94.07%	93.30%	81.58%
15	94.66%	94.04%	93.66%	81.57%

Tabella 3.4: Prestazioni di KNN al variare del numero dei vicini

Tramite osservazioni empiriche, i valori di default dei parametri del metodo del calcolo della distanza e il peso dei vicini hanno ottenuto valori migliori in termini di prestazioni rispetto a valori diversi da quelli di default.

Dopo aver analizzato e trovato i migliori iperparametri, applichiamo una *k*-fold cross-validation per verificare che il modello sia robusto. Il grafico in figura 3.17 mostra che le metriche principali non presentano valori outlier eccessivamente lontani dalla media, si parla di valori che si discostano di un tasso approssimativo dell'1%, questo ci suggerisce che il modello è molto robusto.

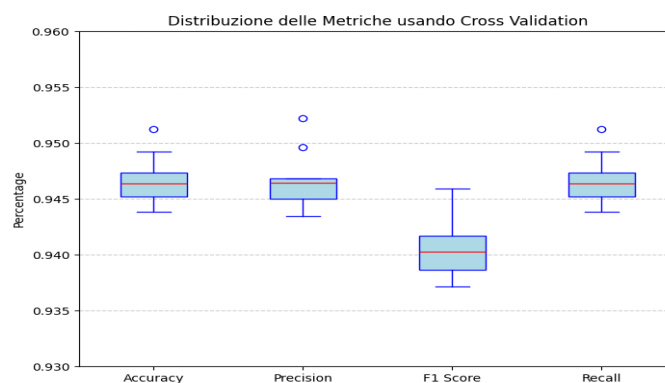


Figura 3.17: Analisi della distribuzione delle metriche usando cross validation.

3.4.2 Explainability

Lime

La disamina delle scelte effettuate dal modello mediante **Lime** illustrata in **Figura 3.18** mette in luce quali features siano state più impattanti durante la fase di predizione. Le prime due variabili indipendenti per rilevanza decisionale sono: HbA1c e glucose, sottolineando la bontà del modello, ma con una scarsa divisione per i valori di **glucose** dividendo le due classi per valori glicemici superiori a 160, dove questo valore soglia è già altissimo per un essere umano a digiuno.

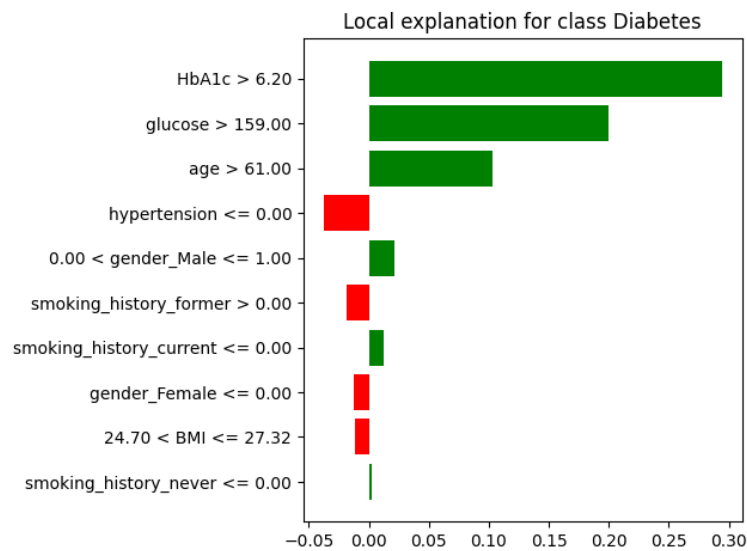


Figura 3.18: knn lime

Shap

Il grafico di un **falso negativo** mostrato in **Figura 3.19** ha messo in evidenza le criticità di un modello basato sulle distanze.

Il modello dà una rilevanza maggiore alle features chiave nella suddetta istanza, stabilendo una giusta relazione fra le variabili, capendo quale sia la corretta scala gerarchica in termini di rilevanza nel contesto del problema, ma è altresì vero che il modello abbia attribuito per un valore di **glucose: 160** un impatto positivo verso la classe negativa, il che può essere dato dal funzionamento del modello in questione, andando così a minare la predizione.

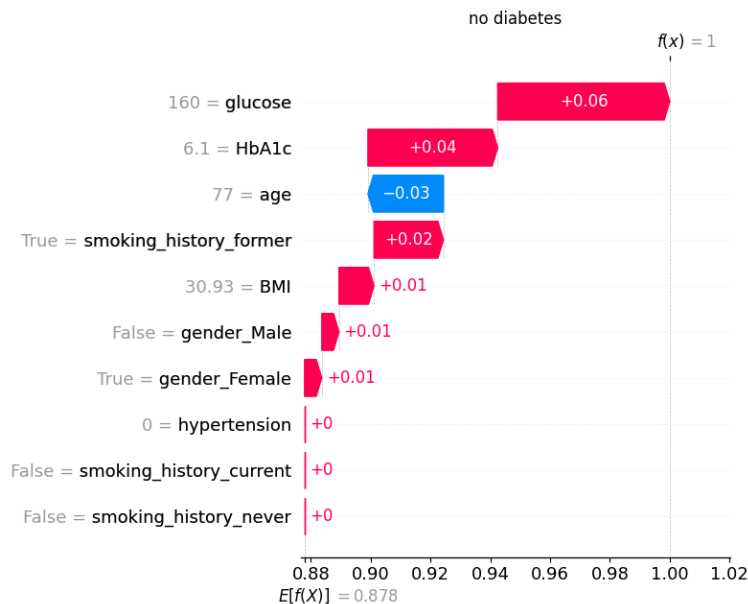


Figura 3.19: Grafico waterfall mostrante un falso negativo di KNN

Essendo state analizzate altre istanze come il **vero positivo** illustrato in **Figura 3.20**, si ribadisce quanto detto. Si evince un corretto "ragionamento" per KNN, cogliendo le giuste relazioni inerenti al problema e dandogli il giusto peso.

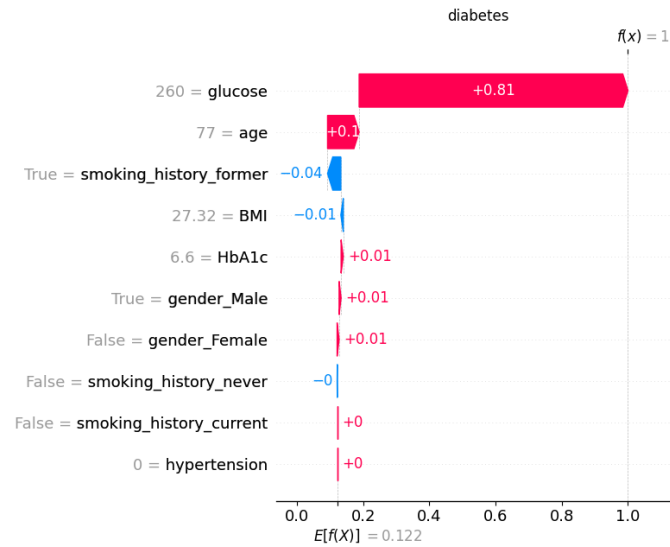


Figura 3.20: Grafico waterfall mostrante un vero positivo di KNN

La discussione del comportamento globale di KNN (**Figura 3.21**) non è molto dissimile dalle precedenti, le features chiave svettano nella scala delle features più importanti, rimarcando una bassa influenza delle features, quali: hypertension, smoking history e gender. Con il crescere dei valori di $f(x)$ vi è una tendenza da parte di **HbA1c e glucose**, da un forte contributo per la classe negativa - blu - a un contributo tendente alla classe positiva - rosso -.

Il modello, in sostanza, non presenta comportamenti anomali da registrare, confermando di aver catturato la relazione del problema.

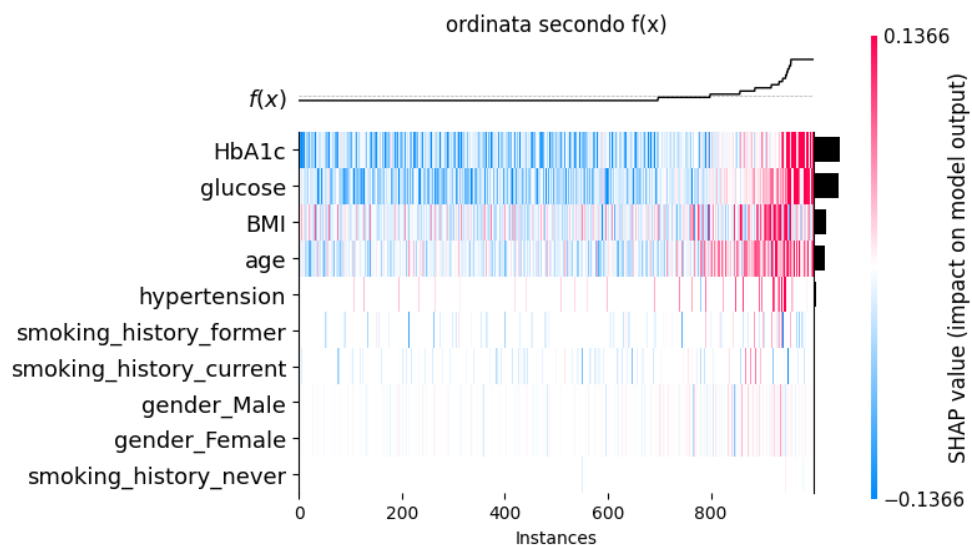


Figura 3.21: Heatmap shap rappresentante il comportamento del modello KNN

3.5 AdaBoostClassifier

3.5.1 Iperparametri

Come spiegato nei modelli precedenti, le prestazioni dei modelli dipendono non solo dal dataset, ma anche da una rifinitura dei parametri di quest'ultimo. Siccome **AdaBoostClassifier** presenta come parametri chiave, solamente, **n_estimators** e **learning_rate**, procediamo col testare tutti i modelli con le coppie (x, y) , appartenenti al prodotto cartesiano dei due insiemi, così definiti:

- **N-estimators:** $Est = \{n \in \mathbb{N} \mid n = 10k : 5 \leq k \leq 19, k \in \mathbb{N}\}$
- **Learning rate:** $Lrn = \{x \in \mathbb{R} \mid x = k \cdot 10^{-1} : 1 \leq k \leq 19, k \in \mathbb{N}\}$

Testando tutte le coppie (x, y) del prodotto cartesiano $Est \times Lrn$ possiamo visualizzare l'andamento delle prestazioni del modello, al variare di esse. Con l'ausilio dei grafici in **Figura 3.22**, siamo in grado di ottenere una panoramica delle coppie di parametri più promettenti.

Partiamo dal primo grafico, che riguarda la precisione di **ADA** per valori inferiori a 1, che costituisce il valore di default del modello; si ha una precisione del 100%, il che ci porta a pensare che il modello soffrirà di overfitting, visto che nella fase di tuning non usiamo la **k-cross-validation**, interpretiamo questi valori come un modello scorretto. Per quanto riguarda il numero di estimatori, non si osservano dati rilevanti, il modello ottiene pressoché prestazioni simili al variare di essi, in generale si attestano ottimi valori per quanto riguarda la **F1-score e Recall**. Un dato che si evince sia dal grafico della precisione sia dai restanti tre, è la presenza di un plateau nelle prestazioni del modello con valori inferiori a 1 della **learning rate**, quindi scarteremo le coppie che presentano questi valori. Questa tesi è corroborata dall'analisi del modello tramite **average precision score e Area under curve (AUC)**, mostrati in **Figura 3.23**.

Il primo dei due grafici ci aiuta a capire quanto il modello riesca a dividere correttamente le due classi, in questo caso, il modello parte da valori intorno al 85% per poi salire fino al 88% quando il **learning rate** si appresta intorno a 1.3 e un numero

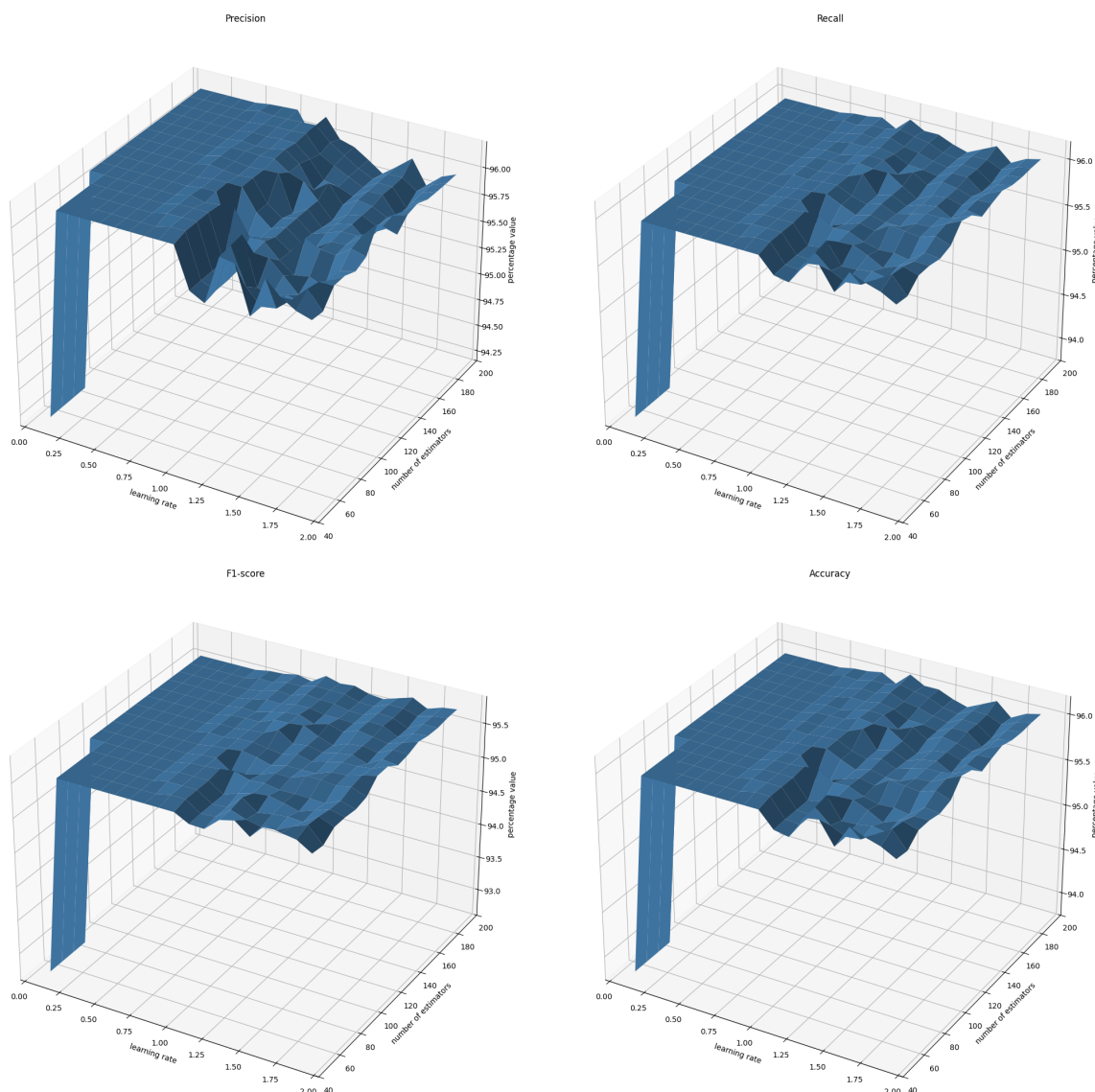


Figura 3.22: Grafici 3d che rappresentano l'andamento delle metriche f1-score, recall, precision e accuracy, al variare della learning rate e n-estimators

di estimatori superiori a 100, con un calo delle prestazioni con valori di **learning rate** troppo alti associati a un numero troppo basso di **estimatori**. Il secondo grafico ci può aiutare a capire quanto bene il modello riesca a predire i casi positivi al diabete, visto che il dataset è sbilanciato. Possiamo osservare un trend crescente date le condizioni poste in precedenza, anche in questo caso i valori ottimali si riscontrano tra valori di **learning rate** di un intorno di 1.3 ed un numero superiore a 100 per quanto riguarda il numero di estimatori.

Dopo aver analizzato i dati, abbiamo scelto di testare il nostro modello con para-

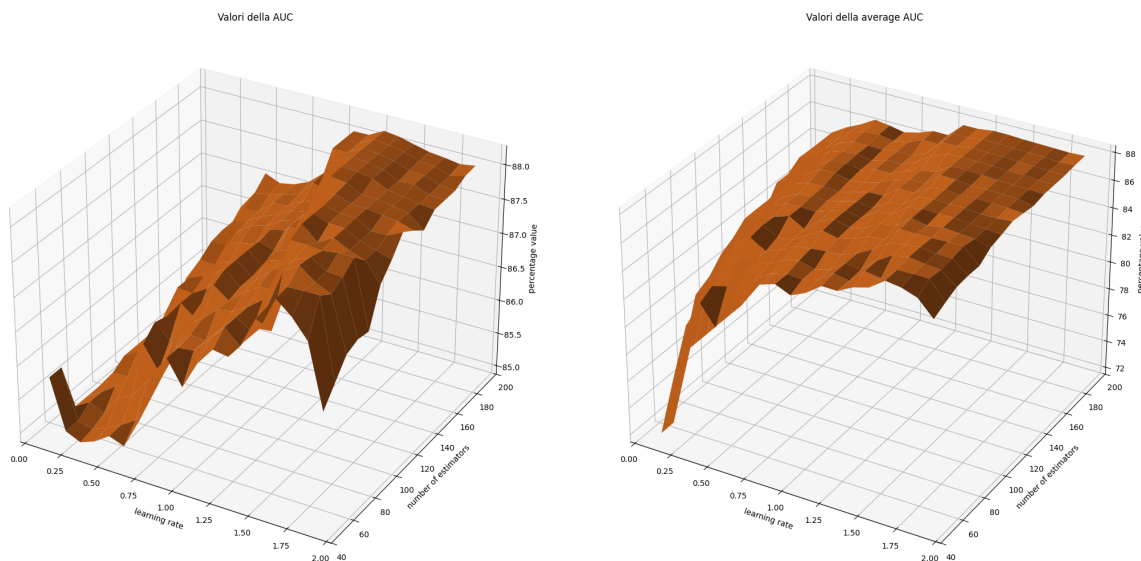


Figura 3.23: Confronto delle prestazioni del modello con parametri di default sulla destra e usando max-depth 10 sulla sinistra usando la K-cross-validation

metri di **learning rate**: 1.25 e **n-estimators**: 160, avendo fatto *fine-tuning* per trovare gli iperparametri più promettenti. Passiamo ora ad analizzarli tramite la solita **k-cross-validation** che abbiamo usato fino ad ora, con $k = 10$, così da poter verificare l'assenza di dati anomali del nostro modello e constatare la sua robustezza.

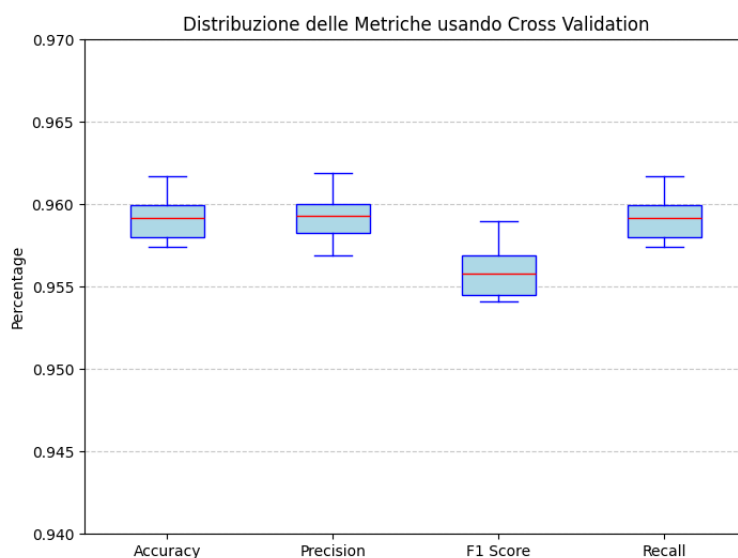


Figura 3.24: Distribuzione delle metriche usate su 10 iterazioni della k-cross-validation

L'utilizzo della **k-cross-validation** ha confermato la robustezza del modello, che

non ha presentato valori outlier e una distribuzione pressappoco normale delle metriche usate, come riportato dalla **Figura 3.24**.

3.5.2 Explainability

Lime

Viene mostrata in **Figura 3.25** una corretta predizione di un non diabetico. Dal grafico ricaviamo come le features chiave siano le più importanti per la predizione, con un valore **HbA1c** inferiore a 4.80 vi è un quasi totale sbilanciamento verso la classe negativa seguita da valori di **glucosio** inferiori alle 100 unità, con influenza quasi nulla per il resto delle features.

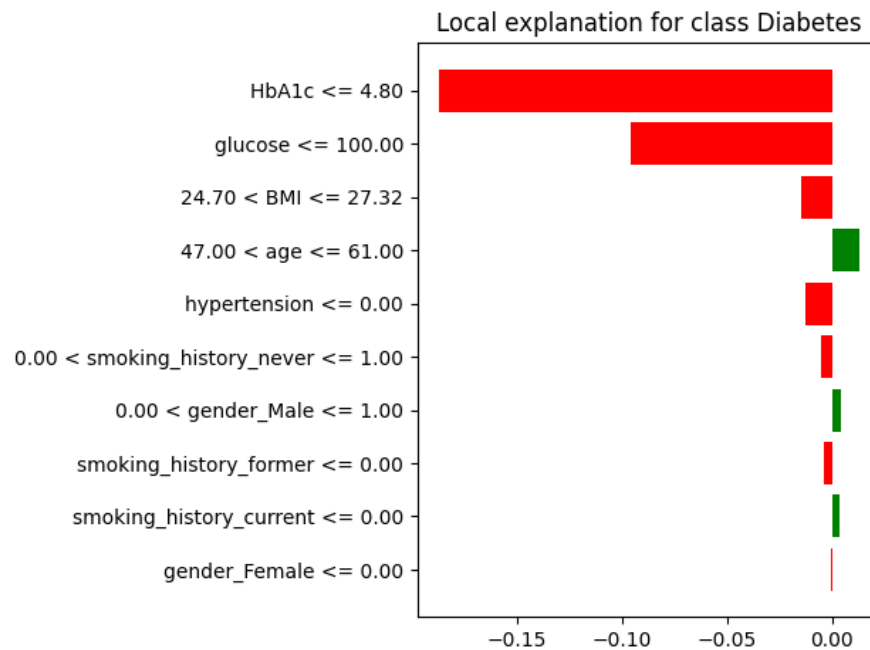


Figura 3.25: Grafico mostrante le scelte di una predizione fatta con ADA tramite lime

Shap

Per quanto riguarda il comportamento di **Adaboost** si mostra una predizione di un **falso negativo** con l'ausilio della **Figura 3.26** analizzata con un waterfall shap, partendo da un valore atteso di 54% per la classe negativa, un valore di **age** e **glucose** alti spingono la predizione verso la classe positiva, leggermente compensata da **HbA1c** che portano il modello verso un 54% per la classe negativa e pregiudicando così la predizione. Nonostante ciò, il modello pesa correttamente le features ponendo in alto quelle più discriminanti.

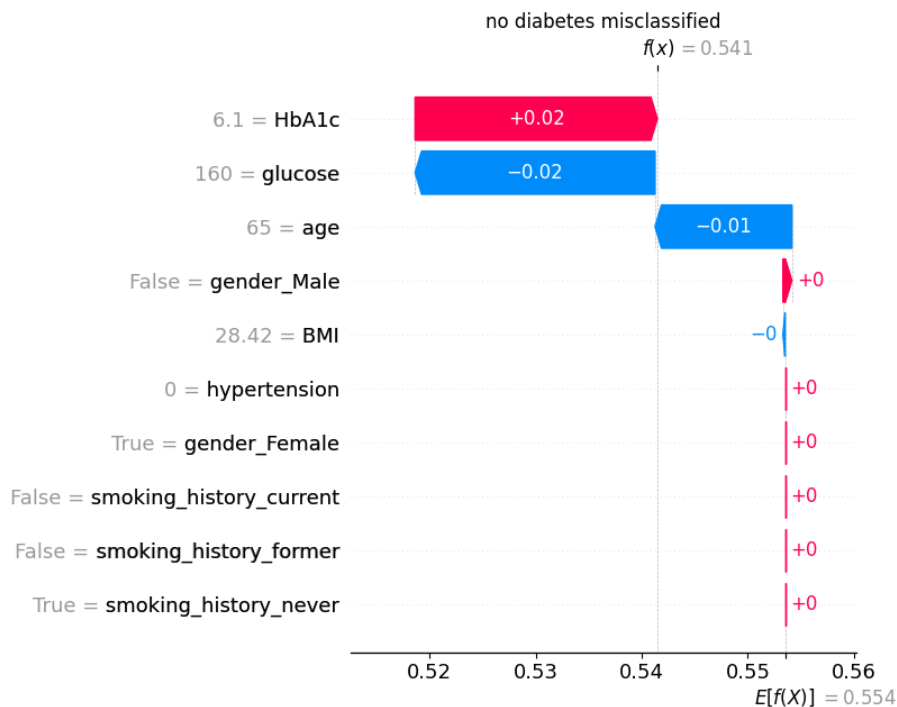


Figura 3.26: Grafico waterfall di un falso negativo di Adaboost

La disamina delle scelte di un **vero positivo** illustrato in **Figura 3.27**, mostra come il risultato finale corretto sia del 51% a favore della classe positiva, con **glucose**, **BMI**, **HbA1c**, che influenza la predizione fino a portare il modello verso la maggioranza del 51% per la classe positiva.

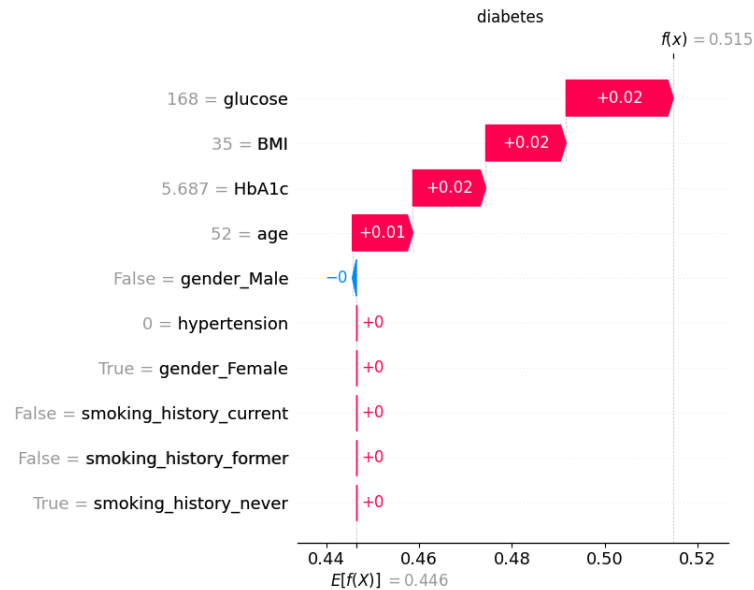


Figura 3.27: Grafico waterfall di un vero positivo di Adaboost

Ciò che si è ricavato invece dalla heatmap(Figura 3.28) del comportamento globale del modello, è analogo a quelli già discussi, le features dominanti sono: **HbA1c, glucose, age e BMI**. Svetta in cima a tutte le altre **HbA1c** secondo **Adaboost**, che con la transizione da classe negativa - valori più bassi di $f(x)$ sinistra - a quella positiva - valori più alti di $f(x)$ destra - vi è un netto passaggio di contributo verso la classe iniziale - blu - a quella finale - rosso - citate sopra, ponendola come variabile quasi assoluta per la discriminazione del diabete.

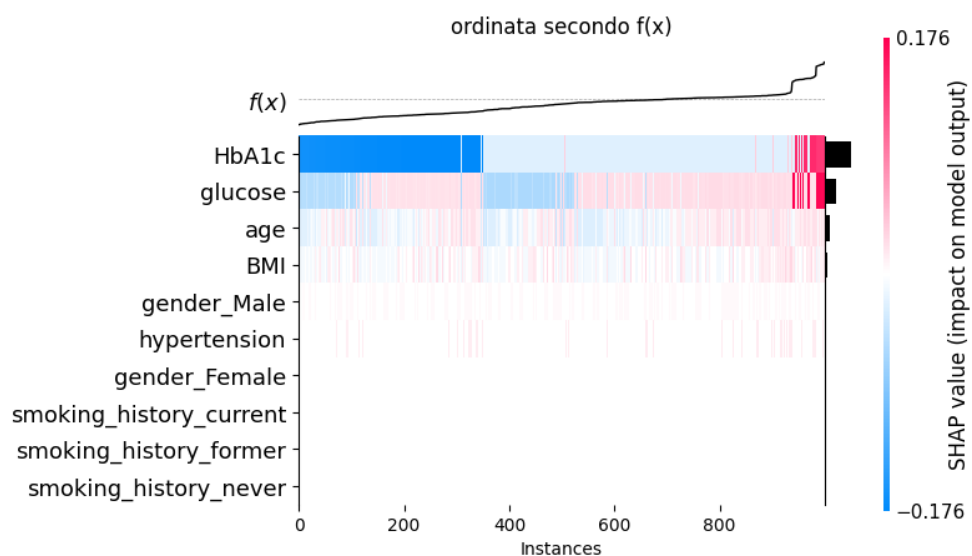


Figura 3.28: Hetmap shap mostrante il comportamento globale di adaboost

3.6 Log Regression

3.6.1 Iperparametri

I parametri scelti per questo modello sono i seguenti: **Solver:** newton-cholesky, **class_weight:** balanced, **c:** 0,3. Prove empiriche non hanno apportato modifiche alle performance del modello, quindi analizziamo tramite la **Figura 3.29**, come esso si comporta con una verifica iterativa dei fold di test. Il modello presenta metriche altalenanti con alcuni outlier e un'alta **precision** a discapito della **recall**, con un valore inferiore del 5%. Visto il target di ricerca, questo modello, valutato solo attraverso le metriche matematiche, risulta essere quello che ha performance peggiori tra tutti i modelli scelti. Dato un valore **AUC** del 80%, e quindi un modello abbastanza buono, si è deciso di non rimuovere questo modello dalla pool dei modelli presenti.

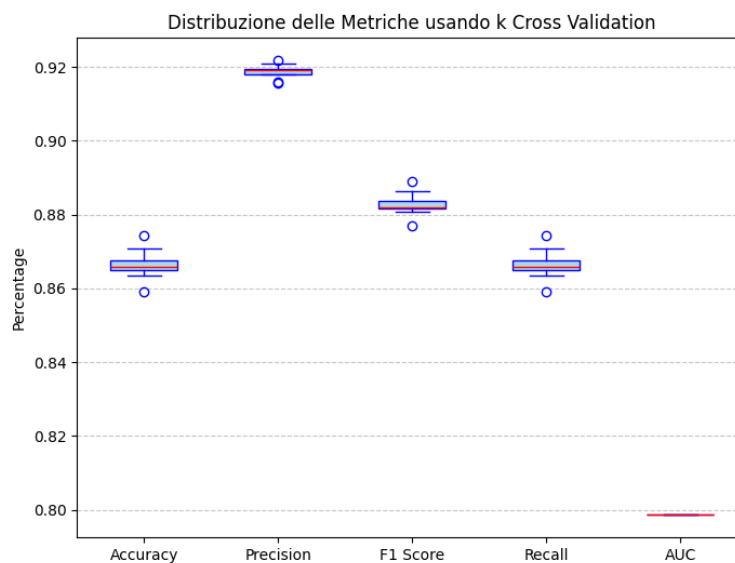


Figura 3.29: Visualizzazione della media delle metriche del modello usando k cross validation con k=10

3.6.2 Explainability

Lime

Il grafico lime presentato in **Figura 3.30** ci evidenzia come il modello etichetti questa predizione come negativa al diabete, poiché assume che i valori **HbA1c** e **glucose** siano i più discriminanti in aggiunta a valori al di sotto di una certa soglia, portando verso la classe negativa.

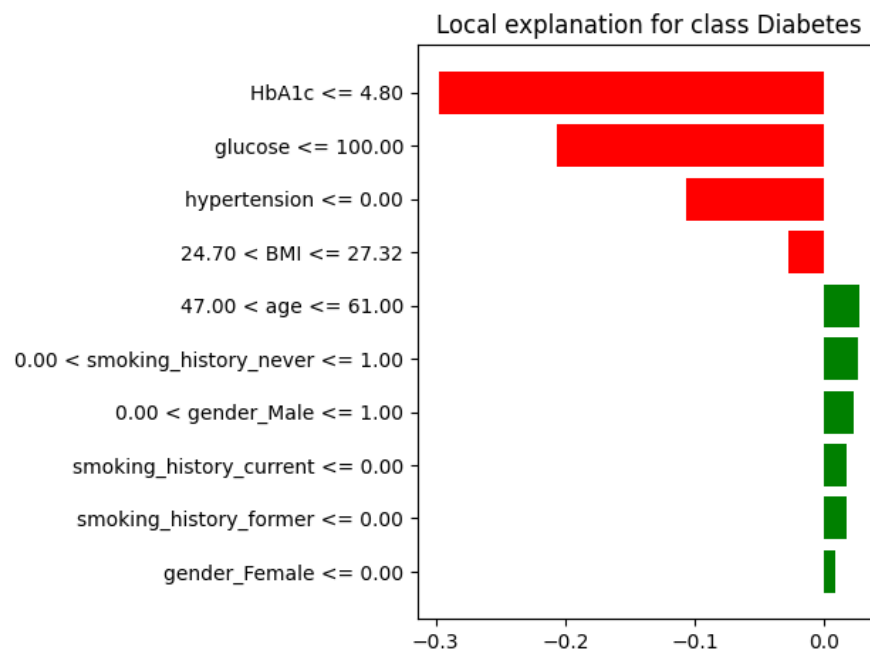


Figura 3.30: Grafico mostrante le scelte di una predizione di log-regression tramite lime

Shap

In merito alle valutazioni shap effettuate, viene mostrato in **Figura 3.31** un **falso negativo** analizzando le scelte effettuate mediante un grafico waterfall. Si ricava un'influenza verso la classe positiva per **HbA1c** che è stata la più impattante come ci si aspetterebbe, ma non sufficientemente da portare la predizione verso la classe corretta che in questo caso è quella positiva, rimarcando quanto già detto, cioè che un modello lineare semplice riscontri delle difficoltà in merito al problema preso in esame.

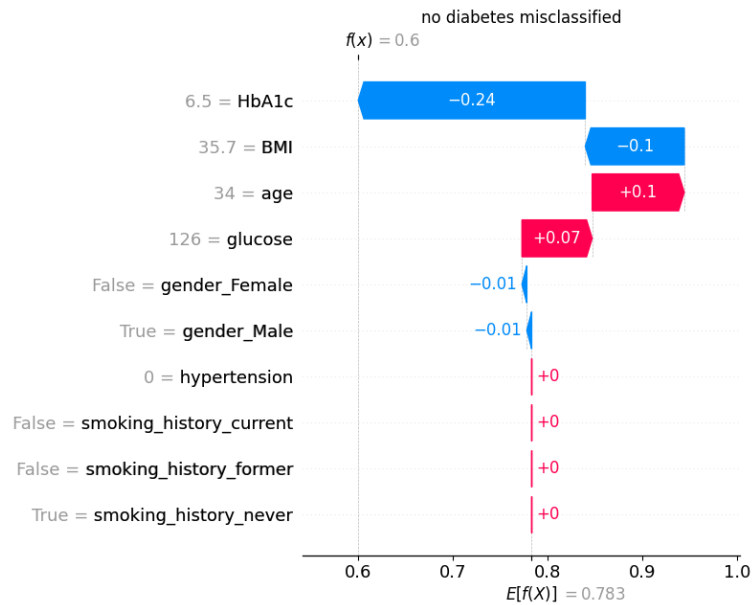


Figura 3.31: Grafico waterfall mostrante un falso negativo di log-regression

L'analisi di un **vero positivo** (Figura 3.32) mostra comunque una discreta qualità del modello viste le sue limitatezze, su casi comunque non lontani dalla regressione.

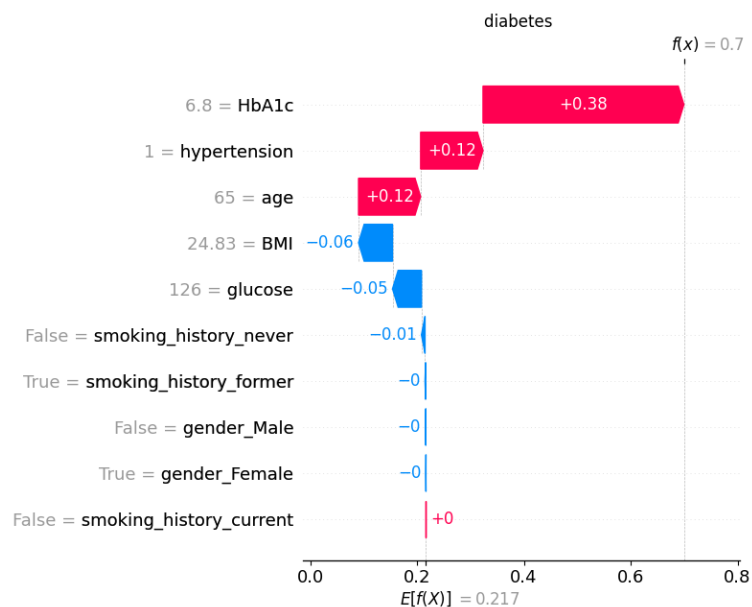


Figura 3.32: Grafico waterfall mostrante un vero positivo di log-regression

In ultima analisi, si è verificato il comportamento globale del modello riportando un grafico in **Figura 3.33**.

Il modello di regressione lineare si allinea ai modelli precedenti, dando peso alle già citate 4 features che spiccano sulla scala di importanza, ma è altresì vero che il comportamento del modello al crescere di $f(x)$ sia molto meno armonioso rispetto ai precedenti, rimarcando i problemi di un regressore lineare. Vi è uno sbilanciamento molto più repentino verso la classe positiva rispetto ai modelli che hanno ottenuto prestazioni migliori.

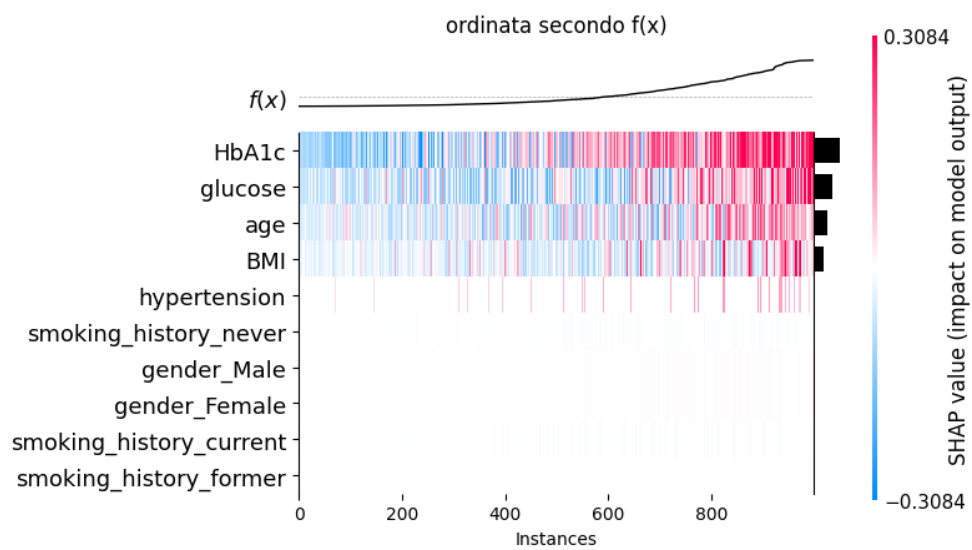


Figura 3.33: Heatmap shap mostrante le scelte generali del modello log-regression

3.7 Riepilogo risultati

Prima di poter effettuare una panoramica dei risultati è importante menzionare i criteri adottati per poter confrontare i modelli fra di loro. Il primo criterio è quello di ordinare secondo le metriche di **F1-score**, **recall** e **AUC-score** (discusse in sezione 2.3) ponendo in cima i modelli che hanno ottenuto valori maggiori. Il secondo criterio è il risultato di un'analisi di ogni modello mediante **shap** e **lime**. Capire a livello locale perché hanno sbagliato i modelli e come le features più importanti hanno influenzato erroneamente la scelta. In aggiunta, una valutazione globale del modello ottenuta tramite grafici **shap** - le heatmap usate per questa ricerca - analizzando eventuali pattern o rumore nella valutazione delle features.

Partendo dal peggiore:

1. il modello che ha ottenuto prestazioni peggiori sia in termini metrici, sia in termini di explainability, è stato **log-regression**. Il modello in questione dà importanza maggiore alle giuste features, ma è presente una forte dispersione e rumore del contributo di ogni features come si è osservato nel grafico 3.33. Anche l'analisi del **falso negativo** 3.31 ci porta a pensare che un regressore lineare non si adatti al problema visto il poco contributo di **HbA1c** alla classe positiva esame e un valore verso la classe negativa per **glucose** avente valore 126. Per i motivi elencati, riteniamo questo modello come il peggiore fra i 6 modelli scelti per questa ricerca.
2. **KNN**; essendo il penultimo in termini di performance in aggiunta alle seguenti motivazioni.

Quanto osservato durante l'analisi di un **falso negativo** (Figura 3.19), evidenzia i limiti che possono scaturire dal **KNN** che si basa su distanza, con una probabilità del 100% è stato classificato un individuo con 160 di glucose e 6.1 di HbA1c come negativo al diabete al 100%. Con un valore di glucosio di 160 che risulta essere ampiamente fuori norma, **KNN** ha ritenuto questo valore positivamente discriminate per la classe negativa, il che ci porta a dire che, il

risultato sia scaturito dalla limitatezza di un modello che si basa su distanze come **KNN**.

Essi risultano essere i peggiori due modelli riscontrati.

Da qui in poi ci limiteremo alle sole valutazioni dei comportamenti dei modelli, siccome tutti al di fuori di **log-regression** e **knn** hanno ottenuto pressoché un punteggio simile in termini metrici.

Si discute di seguito in modo congiunto **Random forest**, **Xgboost** e **AdaClassifier**, lasciando alla fine le considerazioni riguardanti **MLP**.

Tutti i modelli sopra menzionati non disperdono le loro predizioni su molte features che possono essere rumorose, concentrandosi sulle solite **HbA1c**, **glucose**, **age**, **BMI** ponendo in cima alla scala di rilevanza **HbA1c** e **glucose**. Tutti hanno un trend crescente per la classe positiva associato a un contributo maggiore dato da **HbA1c** e **glucose**, suggerendo come i modelli siano robusti ed equamente validi, così come per le loro pecche.

Sono stati riscontrati quasi gli stessi problemi per i modelli citati, come la poca rilevanza data alle features chiave durante l'analisi delle predizioni errate dove sono presenti valori soglia di **HbA1c** e **glucose**, dove i modelli non analizzano le predizioni considerando le relazioni intrinseche tra alcune features. Detto ciò, tutti i modelli si possono ritenere molto validi nel dare una prospettiva diversa di analisi a supporto delle diagnosi, ponendoli come i migliori 3 modelli della ricerca.

In ultima analisi, si discute **MLP**, che si vuole mettere al di fuori rispetto agli altri modelli, comunque ritenendolo il quarto migliore della ricerca. Nonostante quest'ultimo abbia mostrato una leggera distribuzione della rilevanza su tutte le features andando a sfruttarle tutte, riteniamo che fra tutti, questo modello con un *fine-tuning* corretto o con varianti sfruttanti la base dei **Multi-layer perceptron**, si possa migliorare quanto fatto dagli altri modelli, potendo portare il modello a cogliere le relazioni intrinseche che sono presenti fra le due features chiave (**HbA1c** e

glucose). Un uso congiunto durante la valutazione potrebbe mitigare i falsi negativi che nascono sui valori soglia delle features chiave, prendendo in analisi entrambe le features mettendole in relazione.

CAPITOLO 4

Conclusioni

L'uso di modelli AI in contesti reali è in costante crescita, portando con sé le problematiche di questi modelli, come la scarsa interpretabilità, risultando, così, non utilizzabili in contesti critici come quello del diabete mellito analizzato in questa ricerca.

Questo studio si è quindi posto l'obiettivo di creare una piattaforma software in grado di predire il diabete mediante l'uso di diversi modelli di machine learning, consentendo così un'analisi incrociata e assicurando la trasparenza decisionale grazie alle tecniche di explainability **SHAP** e **LIME**.

Si è quindi scelto un dataset già ingegnerizzato, partendo così da una base solida; vi è stata una fase di **Data Understanding & Data Preparation**, verificando se il dataset dovesse essere sottoposto a ulteriori fasi di ingegnerizzazione, dove si è discusso del perché siano stati tenuti gli outlier presenti e dello sbilanciamento del dataset. Andando poi ad analizzare i dati per rilevare eventuali relazioni presenti.

Terminata la fase di analisi del dataset, si è passati all'implementazione dei 6 modelli di machine learning scelti, quali: **Random forest, Xgboost, multi-layer perceptron, KNN, adaboost e log-regression**. I risultati hanno mostrato come gli algoritmi ensemble (in particolare XGBoost e AdaBoost) abbiano garantito i migliori valori di AUC-ROC ($\approx 0,88\sim 0,89$) e una buona robustezza agli outlier, mentre altri modelli basati su regressione o metriche di distanza (**Log-regression, KNN**) hanno mostrato prestazioni nettamente inferiori.

L'analisi tramite SHAP e LIME ha confermato il ruolo predominante di HbA1c e glucosio nell'orientare le decisioni predittive, riconfermando attraverso le analisi globali le ottime prestazioni ottenute dai migliori modelli. L'approccio proposto consente di garantire alte prestazioni, supportate da tecniche che offrono trasparenza locale e globale dei modelli AI, così da poter accrescere la fiducia nel sistema proposto.

Andiamo ora a presentare quali possano essere gli spunti futuri di questa ricerca.

Un primo ambito di approfondimento potrebbe riguardare l'ampliamento e l'arricchimento del dataset. L'integrazione di dati longitudinali – ovvero misurazioni ripetute su uno stesso paziente nel tempo – permetterebbe non solo una valutazione più accurata dell'evoluzione della malattia, ma anche l'introduzione di modelli predittivi orientati al rischio futuro, in ottica di prevenzione.

Un secondo filone di sviluppo, riguarda l'esplorazione di tecniche o modelli che possano fare uso delle relazioni presenti fra le features, potendo così aumentare l'accuratezza delle predizioni.

Un altro passo potrebbe riguardare la validazione esterna su popolazioni differenti. Solo attraverso una validazione multicentrica sarà possibile dimostrare la reale omogeneità dei modelli e garantirne l'uso in scenari clinici eterogenei.

Bibliografia

- [1] S. F. D. Aoife M Egan, "What is diabetes," *Medicine*, vol. 47, no. 1, p. 1, 2019. [Online]. Available: <https://doi.org/10.1016/j.mpmed.2018.10.002> (Citato alle pagine 1, 2 e 6)
- [2] World Health Organization, "Diabetes," 2024, overview available online. [Online]. Available: <https://www.who.int/en/news-room/fact-sheets/detail/diabetes> (Citato a pagina 2)
- [3] S. Esposito, "diabetes ml prediction," 2024, riferimento al Dataset utilizzato. [Online]. Available: https://github.com/simuan02/diabetes_ml_prediction (Citato a pagina 5)
- [4] IBM, "Che cos'è la foresta casuale?" riferimento a Random forest. [Online]. Available: <https://www.ibm.com/it-it/think/topics/random-forest> (Citato a pagina 14)
- [5] —, "Che cos'è xgboost?" riferimento a Xgboost. [Online]. Available: <https://www.ibm.com/it-it/think/topics/xgboost> (Citato a pagina 15)
- [6] scikit learn, "Neural network models(supervised)," riferimento a mlp. [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html (Citato a pagina 16)

- [7] IBM, “Che cos’è l’algoritmo knn?” riferimento a KNN. [Online]. Available: <https://www.ibm.com/it-it/topics/knn> (Citato a pagina 17)
- [8] S. Baladram, “Adaboost classifier, explained: A visual guide with code examples,” *TDS Archive*, 2024. [Online]. Available: <https://medium.com/data-science/adaboost-classifier-explained-a-visual-guide-with-code-examples-fc0f25326d7b> (Citato a pagina 18)
- [9] scikit learn, “Logisticregression,” riferimento a log-regression. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (Citato a pagina 20)
- [10] C. G. Marco Tulio Ribeiro, Sameer Singh, ““why should i trust you?” explaining the predictions of any classifier,” *arxiv.org*, 2016. [Online]. Available: <https://arxiv.org/abs/1602.04938> (Citato a pagina 21)
- [11] P. Banerjee, “Explain your model predictions with lime,” *Kaggle*, 2020. [Online]. Available: <https://www.kaggle.com/code/prashant111/explain-your-model-predictions-with-lime> (Citato a pagina 21)
- [12] S.-I. L. Scott M. Lundberg, “Aunified approach to interpreting model predictions,” *arxiv.org*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.07874> (Citato a pagina 22)

Ringraziamenti

Vorrei ringraziare tutte le persone a me care, partendo dalla mia ragazza, alla mia famiglia, ai miei amici, a tutte le persone che sono passate nella mia vita, sia nel bene che nel male, a tutte le persone che non ci sono più, con un riferimento speciale ai gattini Viktor e Victoria...

Se sono quello che sono adesso, è grazie a tutte loro, che mi hanno aiutato a crescere.

Alle persone e ai valori a cui mi ispiro:

Naruto per avermi insegnato i valori dell'amicizia, Batman per avermi insegnato l'integrità. In ultimo ma non per importanza, a Falcone e Borsellino, due persone straordinarie che ammiro e rispetto con tutto me stesso.

Aspiro ad essere come loro. Grazie.

*Questa tesi ha contribuito a piantare un albero in Tanzania - Victoria - e in Ghana - Viktor -
tramite il progetto Treedom.*

<https://www.treedom.net/it/user/sesalab/event/sesa-random-forest>